Exercise No. : **1**

Topics Covered : **Control Structures & Patterns**

Date : **11-06-26**

Solve the following problems

Q No.	Question Detail	Level
1	**Count the digits that divide a Number.** Given an integer num, return *the number of digits in num that divide* num. An integer val divides nums if nums % val == 0. Example 1: Input: num = 7 Output: 1 Explanation: 7 divides itself, hence the answer is 1. Example 2: Input: num = 121 Output: 2 Explanation: 121 is divisible by 1, but not 2. Since 1 occurs twice as a digit, we return 2. Example 3: Input: num = 1248 Output: 4 Explanation: 1248 is divisible by all of its digits, hence the answer is 4. Constraints: • 1 <= num <= 10^9 • num does not contain 0 as one of its digits.	**Easy**
2	**Valid Perfect Square:** Given a positive integer num, return true *if* num *is a perfect square or* false *otherwise*. A perfect square is an integer that is the square of an integer. In other words, it is the product of some integer with itself. You must not use any built-in library function, such as sqrt. Example 1:	**Easy**

	Input: num = 16 Output: true Explanation: We return true because 4 * 4 = 16 and 4 is an integer. Example 2: Input: num = 14 Output: false Explanation: We return false because 3.742 * 3.742 = 14 and 3.742 is not an integer. Constraints: • 1 <= num <= 2^{31} - 1	
3.	**Find the Pivot Integer** Given a positive integer n, find the **pivot integer** x such that: • The sum of all elements between 1 and x inclusively equals the sum of all elements between x and n inclusively. Return *the pivot integer* x. If no such integer exists, return -1. It is guaranteed that there will be at most one pivot index for the given input. **Example 1:** **Input:** n = 8 **Output:** 6 **Explanation:** 6 is the pivot integer since: 1 + 2 + 3 + 4 + 5 + 6 = 6 + 7 + 8 = 21. **Example 2:** **Input:** n = 1 **Output:** 1 **Explanation:** 1 is the pivot integer since: 1 = 1. **Example 3:** **Input:** n = 4 **Output:** -1 **Explanation:** It can be proved that no such integer exist.	**Easy**
4.	You are given an integer n. The score of n is defined as the sum of d * freq(d) over all distinct digits d, where freq(d) denotes the number of times the digit d appears in n. Return an integer denoting the score of n.	**Easy**

	Example 1: Input: n = 122 Output: 5 Explanation: • The digit 1 appears 1 time, contributing 1 * 1 = 1. • The digit 2 appears 2 times, contributing 2 * 2 = 4. • Thus, the score of n is 1 + 4 = 5. Example 2: Input: n = 101 Output: 2 Explanation: • The digit 0 appears 1 time, contributing 0 * 1 = 0. • The digit 1 appears 2 times, contributing 1 * 2 = 2. • Thus, the score of n is 2. Constraints: • 1 <= n <= 10^9	
5.	**Find k-beauty of a number** **Problem Statement :** The k-beauty of an integer num is defined as the number of substrings of num when it is read as a string that meet the following conditions: • It has a length of k. • It is a divisor of num. Given integers num and k, return the k-beauty of num. Note: • Leading zeros are allowed. • 0 is not a divisor of any value. A substring is a contiguous sequence of characters in a string. **Example 1:** Input: num = 240, k = 2 Output: 2 Explanation: The following are the substrings of num of length k: - "24" from "240": 24 is a divisor of 240. - "40" from "240": 40 is a divisor of 240. Therefore, the k-beauty is 2.	**Hard**

Example 2:

Input: num = 430043, k = 2

Output: 2

Explanation: The following are the substrings of num of length k:

- "43" from "430043": 43 is a divisor of 430043.
- "30" from "430043": 30 is not a divisor of 430043.
- "00" from "430043": 0 is not a divisor of 430043.
- "04" from "430043": 4 is not a divisor of 430043.
- "43" from "430043": 43 is a divisor of 430043.

Therefore, the k-beauty is 2.

Constraints:

1 <= num <= 10^9

1 <= k <= num.length (taking num as a string)

Patterns

6 Given a positive integer **N**, print a pattern of stars (*) such that every two consecutive rows contain the same number of stars, and the number of stars increases by 2 after every pair of rows. **Easy**

Pattern Rules

1. The pattern consists of **N** rows.
2. Rows 1 and 2 contain **2 stars** each.
3. Rows 3 and 4 contain **4 stars** each.
4. Rows 5 and 6 contain **6 stars** each.
5. This pattern continues until **N** rows are printed.
6. Stars in a row should be separated by a single space.

Example 1

Input:

6

Output:

```
* *
* *
* * * *
* * * *
* * * * * *
* * * * * *
```

Example 2

Input:

5

Output:

```
* *
* *
* * * *
* * * *
* * * * * *
```

Constraints

1 ≤ N ≤ 100

Explanation

For every pair of rows, the number of stars increases by 2. Therefore, rows (1,2) have 2 stars, rows (3,4) have 4 stars, and rows (5,6) have 6 stars.

7 | **Problem Statement: Inverted Centered Star Pyramid** | **Easy**

Given a positive integer **N**, print an inverted centered star pyramid pattern. The first row should contain **(2 × N - 1)** stars, and each subsequent row should contain **2 fewer stars** than the previous row. The stars should be centered by adding leading spaces before each row.

Pattern Rules

1. The pattern consists of **N** rows.
2. The first row contains **(2 × N - 1)** stars.
3. Each next row contains **2 fewer stars** than the previous row.
4. The stars should be centered by increasing the leading spaces by one position in each row.
5. Stars in a row should be separated by a single space.

Example 1

Input:

5

Output:

```
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

Example 2

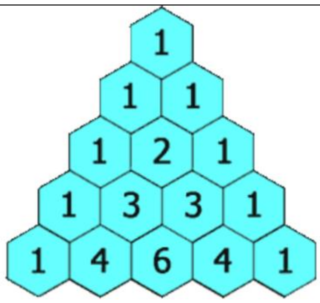
	Input:
	4
	Output:
	* * * * * * *
	  * * * * *
	    * * *
	      *
	Constraints
	1 ≤ N ≤ 100
	Explanation
	For **N = 5**:
	• Row 1 contains **9 stars** (2 × 5 − 1).
	• Row 2 contains **7 stars**.
	• Row 3 contains **5 stars**.
	• Row 4 contains **3 stars**.
	• Row 5 contains **1 star**.
	The number of stars decreases by **2** in each row, while the indentation increases to keep the pattern centered.	

8	**Rectangular numbers** **Problem statement :** Print the pattern in such a way that the outer rectangle is of the number 'N' and the number goes on decreasing as we move inside the rectangles. For **example,** if 'N' = 4, then pattern will be: 4 4 4 4 4 4 4 4 3 3 3 3 3 4 4 3 2 2 2 3 4 4 3 2 1 2 3 4 4 3 2 2 2 3 4 4 3 3 3 3 3 4 4 4 4 4 4 4 4 . **Sample Input 1:** 2 2 1 **Sample Output 1:**	**Meduim**

<table>
<tr>
<td></td>
<td>2 2 2
2 1 2
2 2 2
1
Explanation Of Sample Input 1:
Test case 1:
For the first test case of sample output 1, as the number is 2, so the outermost rectangle is of number 2. The moment we get inside the rectangle, we reduce the number by 1 and make another rectangle.
Test case 2:
For the second test case of sample output 1, as the number is 1, so the outermost rectangle is of number 1.

Sample Input 2:
1
4
Sample Output 2:
4 4 4 4 4 4 4
4 3 3 3 3 3 4
4 3 2 2 2 3 4
4 3 2 1 2 3 4
4 3 2 2 2 3 4
4 3 3 3 3 3 4
4 4 4 4 4 4 4
Explanation Of Sample Input 2:
Test case 1:
For the first test case of sample output 2, as the number is 4, so the outermost rectangle is of number 24. The moment we get inside the rectangle, we reduce the number by 1 and make another rectangle. This process goes on till we reach 1.

Constraints:
1 <= T <= 5
1 <= N <= 100</td>
<td></td>
</tr>
<tr>
<td>9</td>
<td>Given a positive integer n, return the n^{th} row of pascal's triangle.
Pascal's triangle is a triangular array of the binomial coefficients formed by summing up the elements of previous row.</td>
<td>Meduim</td>
</tr>
</table>



Examples:

Input: n = 4

Output: [1, 3, 3, 1]

Explanation: 4^{th} row of pascal's triangle is [1, 3, 3, 1].

Input: n = 5

Output: [1, 4, 6, 4, 1]

Explanation: 5^{th} row of pascal's triangle is [1, 4, 6, 4, 1].

Input: n = 1

Output: [1]

Explanation: 1^{st} row of pascal's triangle is [1].

Constraints:

1 ≤ n ≤ 30

10	**Void of diamond**	**Meduim**

Problem statement : You are given an integer ‘N’, ‘N’ will always be an odd integer. Your task is to print a pattern with the following description:

1. The pattern will consist of ‘N’ lines.
2. The pattern will consist of ‘ ‘ (space) and ‘*’ characters only.
3. The pattern will be a “Void of Diamond” pattern.
4. A “Void of Diamond” pattern is a pattern ‘N’ * ‘N’ cells and ‘ ‘ characters make a diamond shape and ‘*’ fill all other points.
5. For a better understanding of the “Void of Diamond” pattern refer to example and sample input-output.

For example:

If ‘N’ is 5 then the pattern will be-

```
*****
** **
*   *
```

```
** **
*****
```

Sample Input 1:

2

5

3

Sample Output 1:

```
*****
** **
*   *
** **
*****

***
* *
***
```

Explanation of Sample Input 1:

Test Case 1:

Given 'N' = 5

We will print the pattern as the description of the "Void of Diamond" pattern.

Test Case 2:

Given 'N' = 3

There will be only 1 ' ' space in the diamond pattern.

Sample Input 2:

2

7

9

Sample Output 2:

```
*******
*** ***
**   **
*     *
**   **
*** ***
*******
```

```
*********
**** ****
***   ***
**     **
*       *
**     **
***   ***
**** ****
*********
```

Explanation of Sample Input 2:

Test Case 1:

Given ‘N' = 7

The pattern is printed such that ‘ ‘ making a diamond and ‘*’ filling other points.

Test Case 2:

Given ‘N’ = 9

Created a 9*9 grid and all space cells make a diamond pattern.

Exercise No. **: 2**

Topics Covered **: Arrays_Strings**

Date **: 11-06-26**

Solve the following problems

Q No.	**Question Detail**	**Level**
1	Two Sum Given an array of integers nums and an integer target, return *indices of the two numbers such that they add up to target*. You may assume that each input would have *exactly* one solution, and you may not use the *same* element twice. You can return the answer in any order. Example 1: Input: nums = [2,7,11,15], target = 9 Output: [0,1] Explanation: Because nums[0] + nums[1] == 9, we return [0, 1]. Example 2: Input: nums = [3,2,4], target = 6 Output: [1,2] Example 3: Input: nums = [3,3], target = 6 Output: [0,1] Constraints: • $2 <= nums.length <= 10^4$ • $-10^9 <= nums[i] <= 10^9$ • $-10^9 <= target <= 10^9$ • Only one valid answer exists.	**Easy**
2.	Product of Array Except Self Given an integer array nums, return *an array* answer *such that* answer[i] *is equal to the product of all the elements of* nums *except* nums[i].	**Meduim**

	The product of any prefix or suffix of nums is **guaranteed** to fit in a **32-bit** integer. You must write an algorithm that runs in O(n) time and without using the division operation. **Example 1:** **Input:** nums = [1,2,3,4] **Output:** [24,12,8,6] **Example 2:** **Input:** nums = [-1,1,0,-3,3] **Output:** [0,0,9,0,0] **Constraints:** • 2 <= nums.length <= 10^5 • -30 <= nums[i] <= 30 • The input is generated such that answer[i] is **guaranteed** to fit in a **32-bit** integer.	
3.	**Valid Anagram** Given two strings s and t, return true if t is an anagram of s, and false otherwise. **Example 1:** **Input:** s = "anagram", t = "nagaram" **Output:** true **Example 2:** **Input:** s = "rat", t = "car" **Output:** false **Constraints:** • 1 <= s.length, t.length <= 5 * 10^4 • s and t consist of lowercase English letters	**Easy**
4.	Valid Palindrome A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers. Given a string s, return true *if it is a palindrome, or* false *otherwise*.	**Easy**

	Example 1: Input: s = "A man, a plan, a canal: Panama" Output: true Explanation: "amanaplanacanalpanama" is a palindrome. Example 2: Input: s = "race a car" Output: false Explanation: "raceacar" is not a palindrome. Example 3: Input: s = " " Output: true Explanation: s is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome. Constraints: • 1 <= s.length <= 2 * 10^5 • s consists only of printable ASCII characters.	
5	**Rotate String** Given two strings s and goal, return true *if and only if* s *can become* goal *after some number of **shifts** on* s. A **shift** on s consists of moving the leftmost character of s to the rightmost position. • For example, if s = "abcde", then it will be "bcdea" after one shift. **Example 1:** **Input:** s = "abcde", goal = "cdeab" **Output:** true **Example 2:** **Input:** s = "abcde", goal = "abced" **Output:** false **Constraints:** • 1 <= s.length, goal.length <= 100 • s and goal consist of lowercase English letters.	

Exercise No. : **3**

Topics Covered : **Generics & Collections**

Date : **11-06-26**

Solve the following problems

<table>
<tr><th>Q No.</th><th>Question Detail</th><th>Level</th></tr>
<tr><td>1</td><td>Magical Pattern
Problem statement : You have been given an integer 'N'. Your task is to print the Magical Pattern(see examples) for the given 'N'.
Example :
For 'N' : 4
Pattern :
4 3 2 1 2 3 4
3 3 2 1 2 3 3
2 2 2 1 2 2 2
1 1 1 1 1 1 1
2 2 2 1 2 2 2
3 3 2 1 2 3 3
4 3 2 1 2 3 4
Sample Input 1 :
1
3
Sample Output 1 :
3 2 1 2 3
2 2 1 2 2
1 1 1 1 1
2 2 1 2 2
3 2 1 2 3
Sample Input 2 :
1
5
Sample Output 2 :
5 4 3 2 1 2 3 4 5
4 4 3 2 1 2 3 4 4
3 3 3 2 1 2 3 3 3
2 2 2 2 1 2 2 2 2
1 1 1 1 1 1 1 1 1
2 2 2 2 1 2 2 2 2
3 3 3 2 1 2 3 3 3
4 4 3 2 1 2 3 4 4
5 4 3 2 1 2 3 4 5

Constraints :
• 1<= T <= 10
• 1<= N <= 10^2</td><td>Easy</td></tr>
</table>

2	**Interesting Alphabets**	**Easy**

Problem statement : As a part of its competition, the school will conduct a codeathon, Lock the Code, where it has been given a value, and the participants have to decode it.
The participants are given a value denoting the number of rows in the matrix; they need to print the pattern.

Example :
For N=5, Pattern:
E
DE
CDE
BCDE
ABCDE
Among the participants, Ninja is new to programming and doesn't have much experience; he asks you to solve the problem. Can you help solve this problem?

Sample Input 1:
2
5
4

Sample Output 1:
E
DE
CDE
BCDE
ABCDE

D
CD
BCD
ABCD

Explanation for Sample Input 1:
In the first test case, value of 'N' is 5, so print the 'N' rows from 1 to 'N' where in each row start from (N - i - 1)the character which goes on till 'Nth character. Hence the answer is ['E','DE','CDE,' BCDE','ABCDE'].

In the second test case, the value of 'N' is 4, so print the 'N' rows from 1 to 'N' where each row starts from (N - i - 1)the character, which goes on till 'Nth character. Hence the answer is ['D','CD',BCD','ABCD'].

Sample Input 2:
2
3
2

Sample Output 2:
C
BC
ABC

	B AB **Constraints:** 1 <= T <= 50 1 <= N <= 26	
3.	**Word Pattern** **Problem statement :** Given a pattern and a string s, find if s follows the same pattern. Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s. **Example 1:** **Input**: pattern = "abba", s = "dog cat cat dog" **Output:** true **Example 2:** **Input:** pattern = "abba", s = "dog cat cat fish" **Output**: false **Example 3:** **Input:** pattern = "aaaa", s = "dog cat cat dog" **Output:** false **Constraints:** 1 <= pattern.length <= 300 pattern contains only lower-case English letters. 1 <= s.length <= 3000 s contains only lowercase English letters and spaces ' '. s does not contain any leading or trailing spaces. All the words in s are separated by a single space.	**Easy**
4.	**Maximum Product of Two Elements in an Array** **Problem statement :** Given the array of integers nums, you will choose two different indices i and j of that array. Return the maximum value of (nums[i]-1)*(nums[j]-1). **Example 1:** **Input**: nums = [3,4,5,2] **Output**: 12 **Explanation:** If you choose the indices i=1 and j=2 (indexed from 0), you will get the maximum value, that is, (nums[1]-1)*(nums[2]-1) = (4-1)*(5-1) = 3*4 = 12. **Example 2:** **Input:** nums = [1,5,4,5] **Output**: 16 **Explanation:** Choosing the indices i=1 and j=3 (indexed from 0), you will get the maximum value of (5-1)*(5-1) = 16. **Constraints:**	**Easy**

	2 <= nums.length <= 500 1 <= nums[i] <= 10^3	
5	Degree of Array Problem statement : Given a non-empty array of non-negative integers nums, the degree of this array is defined as the maximum frequency of any one of its elements. Your task is to find the smallest possible length of a (contiguous) subarray of nums, that has the same degree as nums. Example 1: Input: nums = [1,2,2,3,1] Output: 2 Explanation: The input array has a degree of 2 because both elements 1 and 2 appear twice. Of the subarrays that have the same degree: [1, 2, 2, 3, 1], [1, 2, 2, 3], [2, 2, 3, 1], [1, 2, 2], [2, 2, 3], [2, 2] The shortest length is 2. So return 2. Example 2: Input: nums = [1,2,2,3,1,4,2] Output: 6 Explanation: The degree is 3 because the element 2 is repeated 3 times. So [2,2,3,1,4,2] is the shortest subarray, therefore returning 6.	**Easy**

	Constraints: nums.length will be between 1 and 50,000. nums[i] will be an integer between 0 and 49,999.	

Exercise No. : **1**

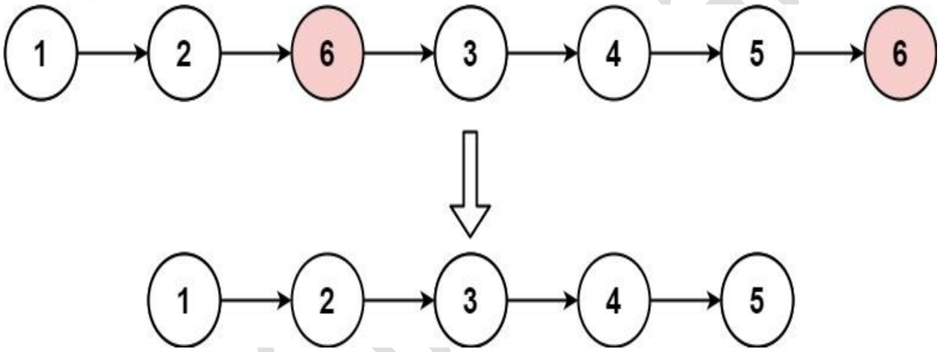
Topics Covered : **Linked List,Stack & Queue**

Date : **12-06-26**

Solve the following problems

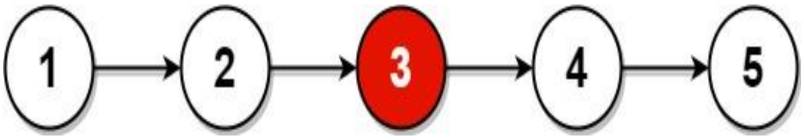
Q No.	Question Detail	Level
1	**Design Linked List** You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: val and next. val is the value of the current node, and next is a pointer/reference to the next node. If you want to use a doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed. Implement the MyLinkedList class: MyLinkedList() Initializes the MyLinkedList object. int get(int index) Returns the value of the index-th node in the linked list. If the index is invalid, return -1. void addAtTail(int val) Appends a node of value val as the last element of the linked list. void addAtIndex(int index, int val) Adds a node of value val before the index-th node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted. void display() Prints all elements of the linked list from head to tail in the format 1 -> 2 -> 3. If the linked list is empty, print "Linked List is empty". **Example 1** **Input** ["MyLinkedList", "addAtTail", "addAtTail", "addAtIndex", "display", "get"] [[], [1], [3], [1, 2], [], [1]] **Output** [null, null, null, null, "1 -> 2 -> 3", 2] **Explanation** MyLinkedList myLinkedList = new MyLinkedList();	**Easy**

	myLinkedList.addAtTail(1);
	myLinkedList.addAtTail(3);
	myLinkedList.addAtIndex(1, 2); // Linked list becomes 1 -> 2 -> 3
	myLinkedList.display(); // Prints: 1 -> 2 -> 3
	myLinkedList.get(1); // Returns 2
	Constraints
	0 <= index <= 1000
	0 <= val <= 1000
	Do not use the built-in LinkedList library.	

2	**Remove Linked List Elements** Given the head of a linked list and an integer val, remove all the nodes of the linked list that has Node.val == val, and return *the new head*. **Example 1:**  **Input:** head = [1,2,6,3,4,5,6], val = 6 **Output:** [1,2,3,4,5] **Example 2:** **Input:** head = [], val = 1 **Output:** [] **Example 3:** **Input:** head = [7,7,7,7], val = 7 **Output:** [] **Constraints:** • The number of nodes in the list is in the range [0, 10^4]. • 1 <= Node.val <= 50 • 0 <= val <= 50	**Easy**
3.	**Middle of the Linked List** Given the head of a singly linked list, return *the middle node of the linked list*. If there are two middle nodes, return **the second middle** node.	**Easy**

<table>
<tr><td></td><td>

Example 1:

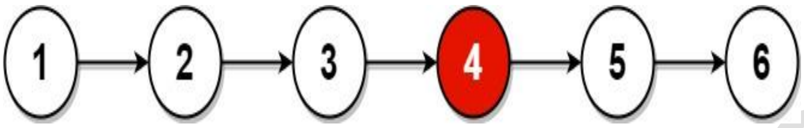


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

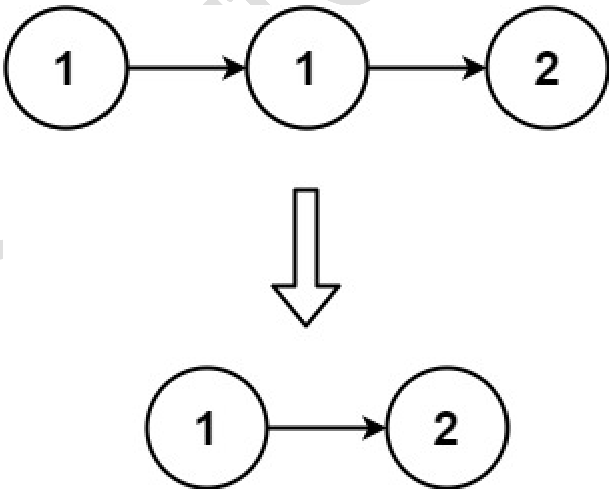
- The number of nodes in the list is in the range [1, 100].
- 1 <= Node.val <= 100

</td><td></td></tr>
<tr><td>4.</td><td>

Remove Duplicates from Sorted List

Given the head of a sorted linked list, *delete all duplicates such that each element appears only once*. Return *the linked list **sorted** as well*.

Example 1:

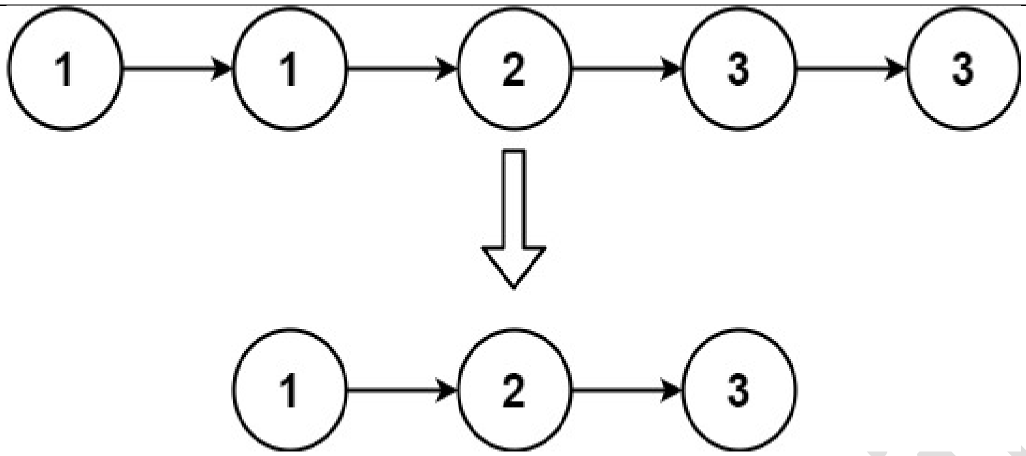


Input: head = [1,1,2]

Output: [1,2]

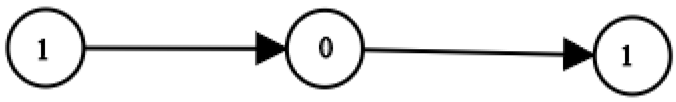
Example 2:

</td><td>Easy</td></tr>
</table>

<table>
<tr>
<td></td>
<td>


Input: head = [1,1,2,3,3]
Output: [1,2,3]
Constraints:

The number of nodes in the list is in the range [0, 300].
-100 <= Node.val <= 100
The list is guaranteed to be sorted in ascending order.

</td>
<td></td>
</tr>
<tr>
<td>5.</td>
<td>
Convert Binary Number in a Linked List to Integer
Given head which is a reference node to a singly-linked list. The value of each node in the linked list is either 0 or 1. The linked list holds the binary representation of a number.
Return the decimal value of the number in the linked list.
The most significant bit is at the head of the linked list.
Example 1:


Input: head = [1,0,1]
Output: 5
Explanation: (101) in base 2 = (5) in base 10
Example 2:
Input: head = [0]
Output: 0
</td>
<td>Easy</td>
</tr>
</table>

	Constraints: • The Linked List is not empty. • Number of nodes will not exceed 30. • Each node's value is either 0 or 1.	

6	**Implement Stack using Queues**
	Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).
	Implement the MyStack class:
	• void push(int x) Pushes element x to the top of the stack.
	• int pop() Removes the element on the top of the stack and returns it.
	• int top() Returns the element on the top of the stack.
	• boolean empty() Returns true if the stack is empty, false otherwise.
	Notes:
	• You must use **only** standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
	• Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.
	
	Example 1:
	Input
	["MyStack", "push", "push", "top", "pop", "empty"]
	[[], [1], [2], [], [], []]
	Output
	[null, null, null, 2, 2, false]
	
	Explanation
	MyStack myStack = new MyStack();
	myStack.push(1);
	myStack.push(2);
	myStack.top(); // return 2
	myStack.pop(); // return 2
	myStack.empty(); // return False
		Easy

	Constraints: • 1 <= x <= 9 • At most 100 calls will be made to push, pop, top, and empty. • All the calls to pop and top are valid.	

7	**Implement Queue using Stacks**
	Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).
	Implement the MyQueue class:
	• void push(int x) Pushes element x to the back of the queue.
	• int pop() Removes the element from the front of the queue and returns it.
	• int peek() Returns the element at the front of the queue.
	• boolean empty() Returns true if the queue is empty, false otherwise.
	Notes:
	• You must use **only** standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
	• Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.
	
	Example 1:
	Input
	["MyQueue", "push", "push", "peek", "pop", "empty"]
	[[], [1], [2], [], [], []]
	Output
	[null, null, null, 1, 1, false]
	
	Explanation
	MyQueue myQueue = new MyQueue();
	myQueue.push(1); // queue is: [1]
	myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
	myQueue.peek(); // return 1
	myQueue.pop(); // return 1, queue is [2]
		Easy

	myQueue.empty(); // return false	
8	**Valid Parentheses** Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. An input string is valid if: 1. Open brackets must be closed by the same type of brackets. 2. Open brackets must be closed in the correct order. 3. Every close bracket has a corresponding open bracket of the same type. **Example 1:** **Input:** s = "()" **Output:** true **Example 2:** **Input:** s = "()[]{}" **Output:** true **Example 3:** **Input:** s = "(]" **Output:** false **Example 4:** **Input:** s = "([])" **Output:** true **Example 5:** **Input:** s = "([)]" **Output:** false **Constraints:** • 1 <= s.length <= 10^4 • s consists of parentheses only '()[]{}'.	**Easy**

9	Reverse Prefix of word Given a **0-indexed** string word and a character ch, **reverse** the segment of word that starts at index 0 and ends at the index of the **first occurrence** of ch (**inclusive**). If the character ch does not exist in word, do nothing. • For example, if word = "abcdefd" and ch = "d", then you should **reverse** the segment that starts at 0 and ends at 3 (**inclusive**). The resulting string will be "dcbaefd". Return *the resulting string*. **Example 1:** **Input:** word = "abcdefd", ch = "d" **Output:** "dcbaefd" **Explanation:** The first occurrence of "d" is at index 3. Reverse the part of word from 0 to 3 (inclusive), the resulting string is "dcbaefd". **Example 2:** **Input:** word = "xyxzxe", ch = "z" **Output:** "zxyxxe" **Explanation:** The first and only occurrence of "z" is at index 3. Reverse the part of word from 0 to 3 (inclusive), the resulting string is "zxyxxe". **Example 3:** **Input:** word = "abcd", ch = "z" **Output:** "abcd" **Explanation:** "z" does not exist in word. You should not do any reverse operation, the resulting string is "abcd". **Constraints:** • 1 <= word.length <= 250 • word consists of lowercase English letters. ch is a lowercase English letter.	**Easy**
10	**Clear Digits** You are given a string s. Your task is to remove **all** digits by doing this operation repeatedly: • Delete the *first* digit and the **closest non-digit** character to its *left*. Return the resulting string after removing all digits.	**Easy**

Note that the operation *cannot* be performed on a digit that does not have any non-digit character to its left.

Example 1:

Input: s = "abc"

Output: "abc"

Explanation:

There is no digit in the string.

Example 2:

Input: s = "cb34"

Output: ""

Explanation:

First, we apply the operation on s[2], and s becomes "c4".

Then we apply the operation on s[1], and s becomes "".

Constraints:

- 1 <= s.length <= 100
- s consists only of lowercase English letters and digits.

  The input is generated such that it is possible to delete all digits.

Hands-on No. **: 1**

Topic **: Java**

Date **: 13-06-2026**

Solve the following problems

Q. No.	Question Detail	Level
1	**Sentence Palindrome** Given a sentence s, determine whether it is a palindrome sentence or not. A palindrome sentence is a sequence of characters that reads the same forward and backward after: • Converting all uppercase letters to lowercase. • Removing all non-alphanumeric characters (i.e., ignore spaces, punctuation, and symbols). Examples: *Input: s = "Too hot to hoot."* *Output: true* *Explanation: If we remove all non-alphanumeric characters and convert all uppercase letters to lowercase, string s will become "toohottohoot" which is a palindrome.* *Input: s = "Abc 012..## 10cbA"* *Output: true* *Explanation: If we remove all non-alphanumeric characters and convert all uppercase letters to lowercase, string s will become "abc01210cba" which is a palindrome.* *Input: s = "ABC $. def01ASDF.."* *Output: false* *Explanation: If we remove all non-alphanumeric characters and convert all uppercase letters to lowercase, string s will become "abcdef01asdf" which is not a palindrome.*	**Easy**

It is going to be hard but, hard does not mean impossible.

2	**Replace all occurrences of substring** Given three strings s, s1, and s2 of lengths n, m, and k respectively, the task is to modify the string s by replacing all the substrings s1 with the string s2 in the string s. Examples: *Input: s = "abababa", s1 = "aba", s2 = "a"* *Output: aba* *Explanation: Change the substrings s[0, 2] and s[4, 6] to the string s2 modifies the string s to "aba".* *Input: s = "geeksforgeeks", s1 = "eek", s2 = "ok"* *Output: goksforgoks* *Explanation: Change the substrings s[1, 3] and s[9, 11] to the string*	**Meduim**
3	**Majority Element** Given an array arr[] of size n, find the element that appears more than ⌊n/2⌋ times. If no such element exists, return -1. Examples: *Input: arr[] = [1, 1, 2, 1, 3, 5, 1]* *Output: 1* *Explanation: Element 1 appears 4 times. Since ⌊7/2⌋ = 3, and 4 > 3, it is the majority element.* *Input: arr[] = [7]* *Output: 7* *Explanation: Element 7 appears once. Since ⌊1/2⌋ = 0, and 1 > 0, it is the majority element.* *Input: arr[] = [2, 13]* *Output: -1* *Explanation: No element appears more than ⌊2/2⌋ = 1 time, so there is no majority element.*	**Easy**
4	**Pairs with sum greater than 0 in an array** Given an array arr[] of size N, the task is to find the number of distinct pairs in the array whose sum is > 0. Examples: *Input: arr[] = { 3, -2, 1 }* *Output: 2*	**Meduim**

It is going to be hard but, hard does not mean impossible.

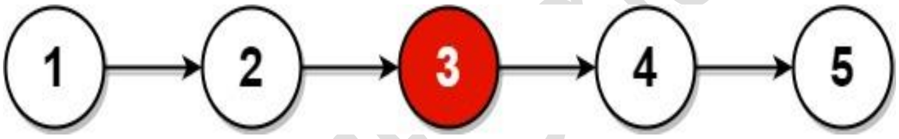
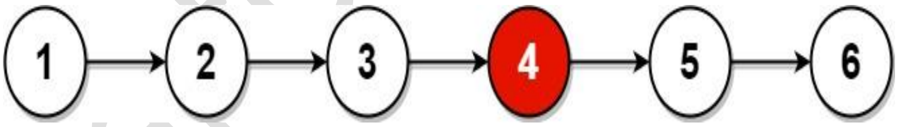
	Explanation: There are two pairs of elements in the array {3, -2}, {3, 1} whose sum is positive. *Input: arr[] = { -1, -1, -1, 0 }* *Output: 0* *Explanation: There are no pairs of elements in the array whose sum is positive.*	
5	Maximum Product Subarray Given an array arr[] consisting of positive, negative, and zero values, find the maximum product that can be obtained from any contiguous subarray of arr[]. Examples: Input: arr[] = [-2, 6, -3, -10, 0, 2] Output: 180 Explanation: The subarray with maximum product is [6, -3, -10] with product = 6 * (-3) * (-10) = 180. Input: arr[] = [-1, -3, -10, 0, 6] Output: 30 Explanation: The subarray with maximum product is [-3, -10] with product = (-3) * (-10) = 30. Input: arr[] = [2, 3, 4] Output: 24 Explanation: For an array with all positive elements, the result is product of all elements.	Meduim

Hands-on No. : **2**

Topic : **Linked List,Stack,Queue**

Date : **13-06-26**

Solve the following problems

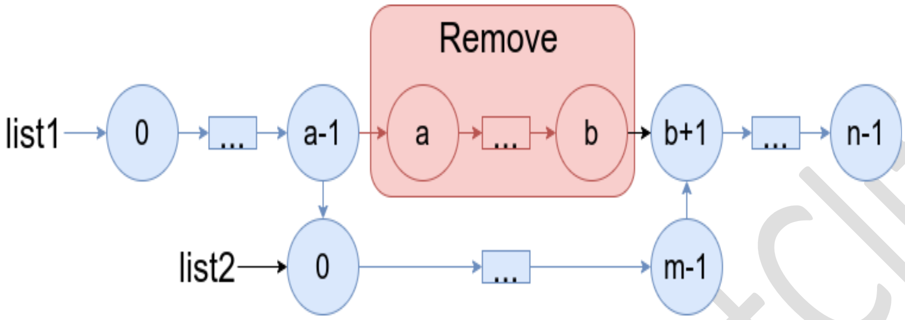
Q. No.	Question Detail	Level
1	**Middle of the Linked List** Given the head of a singly linked list, return *the middle node of the linked list*. If there are two middle nodes, return **the second middle** node. **Example 1:**  **Input:** head = [1,2,3,4,5] **Output:** [3,4,5] **Explanation:** The middle node of the list is node 3. **Example 2:**  **Input:** head = [1,2,3,4,5,6] **Output:** [4,5,6] **Explanation:** Since the list has two middle nodes with values 3 and 4, we return the second one. **Constraints:** The number of nodes in the list is in the range [1, 100]. 1 <= Node.val <= 100	**Easy**

2	**Merge In Between LinkedList**	Easy

You are given two linked lists: list1 and list2 of sizes n and m respectively.

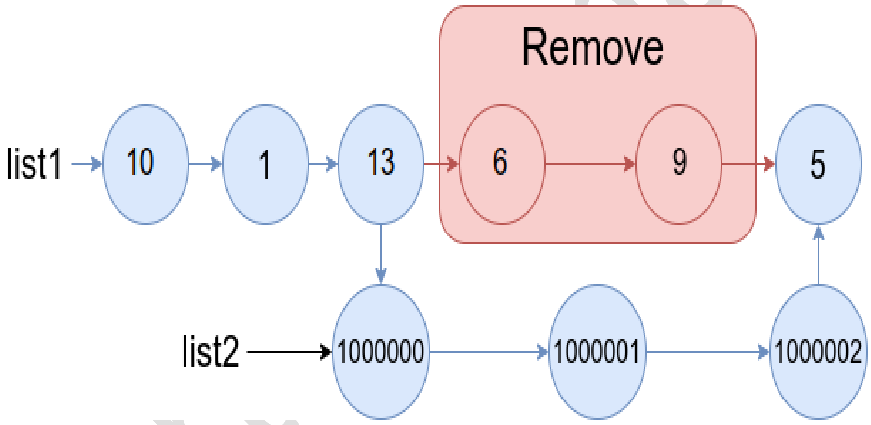
Remove list1's nodes from the a^{th} node to the b^{th} node, and put list2 in their place.

The blue edges and nodes in the following figure indicate the result:



Build the result list and return its head.

Example 1:

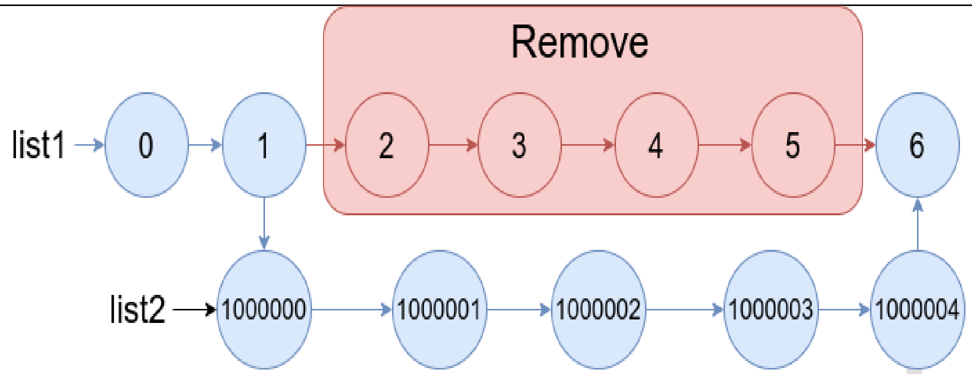


Input: list1 = [10,1,13,6,9,5], a = 3, b = 4, list2 = [1000000,1000001,1000002]

Output: [10,1,13,1000000,1000001,1000002,5]

Explanation: We remove the nodes 3 and 4 and put the entire list2 in their place. The blue edges and nodes in the above figure indicate the result.

Example 2:

	 **Input:** list1 = [0,1,2,3,4,5,6], a = 2, b = 5, list2 = [1000000,1000001,1000002,1000003,1000004] **Output:** [0,1,1000000,1000001,1000002,1000003,1000004,6] **Explanation:** The blue edges and nodes in the above figure indicate the result. **Constraints:** • 3 <= list1.length <= 10^4 • 1 <= a <= b < list1.length - 1 1 <= list2.length <= 10^4	
3	**Reverse the given list** **Problem Statement:** Given the head of a singly linked list, write a program to reverse the linked list, and return the head pointer to the reversed list. **Input**: Linked List: 1 -> 2 -> 3 -> 4 -> 5 **Output**: Reversed Linked List: 5 -> 4 -> 3 -> 2 -> 1 **Constraints :** • 1 <= 'N' <= 10^4 • 0 <= 'data' <= 10^9 Where 'N' is the number of nodes in the linked list.	Easy
4	**Reverse a Sublist of Linked List** **Problem statement** You are given a linked list of 'N' nodes. Your task is to reverse the linked list from position 'X' to 'Y'. **For example:** Assuming, the linked list is 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL. X = 2 and Y = 5. On reversing the given linked list from position 2 to 5, we get 10 -> 50 -> 40 -> 30 -> 20 -> 60 -> NULL.	

For example:

Assuming, the linked list is 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL. X = 2 and Y = 5. On reversing the given linked list from position 2 to 5, we get 10 -> 50 -> 40 -> 30 -> 20 -> 60 -> NULL.

Input 1:

2

10 20 30 40 50 60 -1

2 5

1 2 5 8 9 7 -1

2 4

Output 1:

10 50 40 30 20 60 -1

1 8 5 2 9 7 -1

Explanation of the Input1:

For the first test case, refer to the example explained above.

For the second test case, the linked list is 1 -> 2 -> 5 -> 8 -> 9 -> 7 -> NULL and X = 2 and Y = 4. On reversing the given linked list from position 2 to 4, we get 1 -> 8 -> 5 -> 2 -> 9 -> 7 -> NULL.

Input 2:

2

4 2 5 4 2 2 -1

3 3

1 2 3 4 -1

1 4

Output 2:

4 2 5 4 2 2 -1

4 3 2 1 -1

Constraints:

1 <= T <= 100

1 <= N <= 10^6

1 <= X, Y <= N

1 <= data <= 10^7

<table>
<tr>
<td>5</td>
<td>
Detect a loop in a linked list
Problem Statement : Given the head of a singly linked list, the task is to check if the linked list has a loop. A loop means that the last node of the linked list is connected back to a node in the same list. So if the next of the last node is null. then there is no loop.

Note: You need to return a boolean value true if there is a loop, otherwise false.

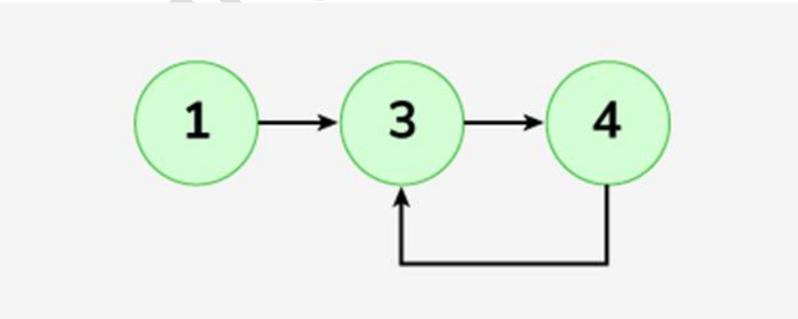
The following is an internal representation of every test case (three inputs).

A LinkedList and a pos (1-based index)-Position of the node to which the last node links back if there is a loop. If the linked list does not have any loop, then pos = 0.

Examples:

Input: LinkedList: 1->3->4

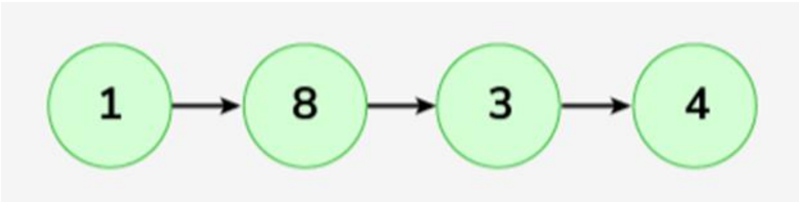
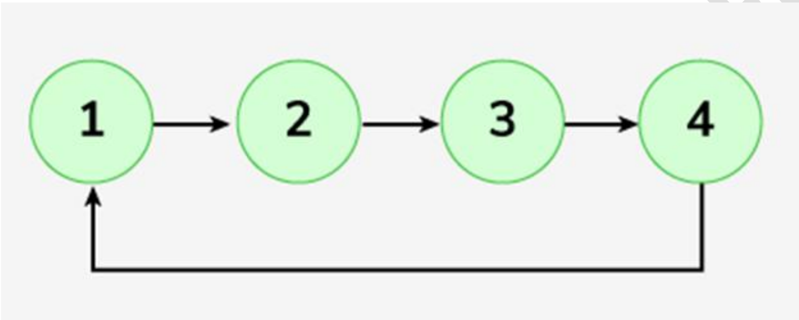
Output: true

Explanation:


See the above list there exists a loop connecting the last node back to the second node.

Input: LinkedList: 1->8->3->4

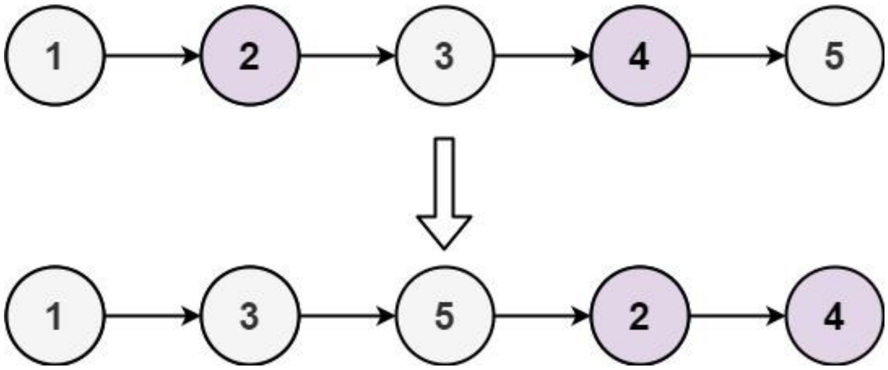
Output: false
</td>
<td>Easy</td>
</tr>
</table>

	Explanation:  There is no loop exists. Input: LinkedList: 1->2->3->4 Output: true Explanation:  A loop is present connecting the last node back to the first node. Expected Time Complexity: O(n) Expected Auxiliary Space: O(1) **Constraints**: 1 ≤ number of nodes ≤ 10^4 1 ≤ node->data ≤ 10^3	
6	**Odd Even Linked List** **Problem statement :**Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*. The first node is considered odd, and the second node is even, and so on.	Easy

It is going to be hard but, hard does not mean impossible.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

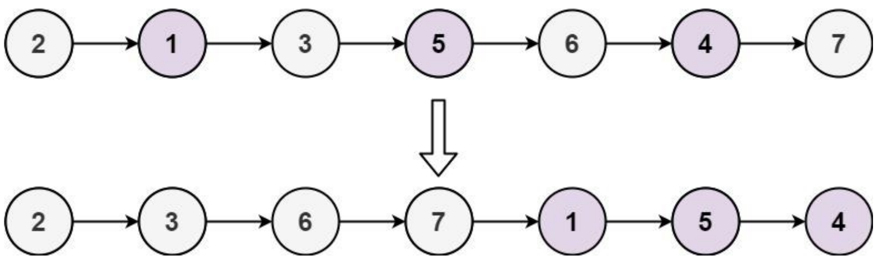
Example 1:



Input: head = [1,2,3,4,5]

Output: [1,3,5,2,4]

Example 2:



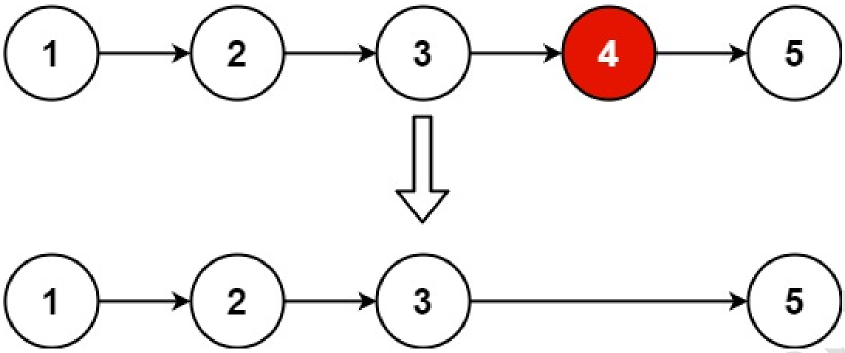
Input: head = [2,1,3,5,6,4,7]

Output: [2,3,6,7,1,5,4]

Constraints:

The number of nodes in the linked list is in the range [0, 10^4].

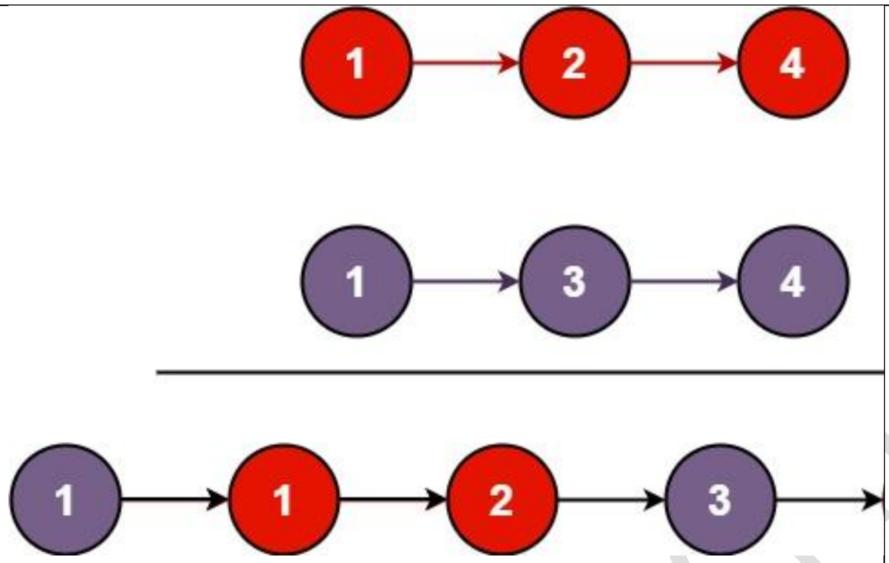
-10^6 <= Node.val <= 10^6

7	**Remove n^{th} node** **Problem Statement:** Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.  **Example 1:** **Input**: head = [1,2,3,4,5], n = 2 **Output:** [1,2,3,5] **Example 2:** **Input:** head = [1], n = 1 **Output:** [] **Example 3:** **Input:** head = [1,2], n = 1 **Output:** [1] **Constraints :** • 1 <= 'N' <= 10^3 • 1 <= 'K' <= 'N' • 1 <= 'data' <= 10^3	Easy
8	**Count Triplets** **Problem statement** You have been given an integer 'X' and a non-decreasing sorted doubly linked list with distinct nodes. Your task is to return the number of triplets in the list that sum up to the value 'X'. **Sample Input 1:** 2 13 -4 2 3 8 9 -1 5 -4 -2 3 4 5 -1	Meduim

	Sample Output 1: 2 2 **Explanation to Sample Input 1:** In the first test case, there are two possible triplets {2,3,8} and {-4,8,9}, whose sum is 13. In the second test case there are two triplets possible {-4,4,5} and {-2,3,4} whose sum is 5. **Sample Input 2:** 2 -4 -3 8 10 -1 12 1 4 6 2 -1 **Sample Output 2:** 0 1 **Explanation to Sample Input 2:** In the first test case, there is no triplet whose sum is -4. In the second test case, there is only one triplet {6,4,2}, whose sum is 12. **Constraints :** 1 <= T <= 5 1 <= N <= 10 ^ 3 - 3 * 10 ^ 5 <= X <= 3 * 10 ^ 5 - 10 ^ 5 <= data <= 10 ^ 5 and data != - 1 Where 'N' is the number of nodes in the given linked list, and 'X' is the required triplet sum value.	
9	**Rotate a Linked List** Given the head of a singly linked list, your task is to left rotate the linked list k times. **Examples**: Input: head = 10 -> 20 -> 30 -> 40 -> 50, k = 4 Output: 50 -> 10 -> 20 -> 30 -> 40	Medium

	Explanation: Rotate 1: 20 -> 30 -> 40 -> 50 -> 10 Rotate 2: 30 -> 40 -> 50 -> 10 -> 20 Rotate 3: 40 -> 50 -> 10 -> 20 -> 30 Rotate 4: 50 -> 10 -> 20 -> 30 -> 40 Input: head = 10 -> 20 -> 30 -> 40 , k = 6 Output: 30 -> 40 -> 10 -> 20 **Constraints**: 1 <= number of nodes <= 10^5 0 <= k <= 10^9 0 <= data of node <= 10^9	
10	**Delete Alternate Nodes** **Problem statement :** Given a Singly Linked List of integers, delete all the alternate nodes in the list. **Example:** List: 10 -> 20 -> 30 -> 40 -> 50 -> 60 -> null Alternate nodes will be: 20, 40, and 60. Hence after deleting, the list will be: Output: 10 -> 30 -> 50 -> null **Input 1:** 1 2 3 4 5 -1 **Output 1:** 1 3 5 **Explanation of Input 1:** 2, 4 are alternate nodes so we need to delete them **Input 2:** 10 20 30 40 50 60 70 -1 **Output 2:** 10 30 50 70	Meduim
11	**Palindrome check**	Meduim

	Problem Statement: You are given the head of a singly linked list. Determine if the linked list is a palindrome, return true if it is a palindrome or false otherwise. A Linked List is a palindrome if it reads the same from left to right and from right to left. **Example**: The lists (1 -> 2 -> 1), (3 -> 4 -> 4-> 3), and (1) are palindromes, while the lists (1 -> 2 -> 3) and (3 -> 4) are not. Sample Input 1: 1 2 2 1 -1 Sample Output 1: True Explanation for Sample Input 1: The given list is a palindrome. Sample Input 2: 3 2 1 -1 Sample Output 2: False **Constraints:** • 1 <= 'N' <= 5 * 10^4 • -10^9 <= 'data' <= 10^9 and 'data' != -1 Where 'N' is the number of nodes in the linked list and 'data' represents the value of the list's nodes.	
12	**Merge Two Sorted List** **You are given the heads of two sorted linked lists list1 and list2.** **Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.** **Return *the head of the merged linked list*.** **Example 1:**	Medium

<table>
<tr><td></td><td>

Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
Example 2:
Input: list1 = [], list2 = []
Output: []
Example 3:
Input: list1 = [], list2 = [0]
Output: [0]

Constraints:
• **The number of nodes in both lists is in the range [0, 50].**
• **-100 <= Node.val <= 100**
• **Both list1 and list2 are sorted in non-decreasing order.**</td><td></td></tr>
<tr><td>**13**</td><td>**Reverse First K elements of Queue**
Problem statement : You are given a QUEUE containing 'N' integers and an integer 'K'. You need to reverse the order of the first 'K' elements of the queue, leaving the other elements in the same relative order.

You can only use the standard operations of the QUEUE STL:

1. enqueue(x) : Adds an item x to rear of the queue
2. dequeue() : Removes an item from front of the queue
3. size() : Returns number of elements in the queue.
4. front() : Finds the front element.</td><td></td></tr>
</table>

For Example:

Let the given queue be { 1, 2, 3, 4, 5 } and K be 3.

You need to reverse the first K integers of Queue which are 1, 2, and 3.

Thus, the final response will be { 3, 2, 1, 4, 5 }.

Sample Input 1:

2

5 3

1 2 3 4 5

4 2

6 2 4 1

Sample Output 1:

3 2 1 4 5

2 6 4 1

Explanation:

For test case 1: Refer to the example explained above.

For test case 2:

The queue after reversing the first 2 elements i.e., 6 and 2 will be { 2, 6, 4, 1 }.

Sample Input 2:

2

5 2

5 3 2 6 4

4 4

1 2 3 4

Sample Output 2:

3 5 2 6 4

4 3 2 1

Constraints:

1 <= T <= 10

1 <= N <= 10 ^ 5

0 <= K <= N

-10 ^ 9 <= queue elements <= 10 ^ 9

14	**Longest Valid Parentheses Substring**	

Problem statement : You are given a string 'STR' consisting only of opening and closing parenthesis i.e. '(' and ')', your task is to find out the length of the longest valid parentheses substring.

Note:

The length of the smallest valid substring '()' is 2.

For example:

'STR' = "()()())" here we can see that except the last parentheses all the brackets are making a valid parenthesis. Therefore, the answer for this will be 6.

Sample Input 1 :

2

()))(())

))((

Sample Output 1 :

4

0

Explanation For Sample Input 1:

For the first test case:

Given 'STR' ="()))(())", we can see that from index 5 to the last, the whole substring is valid. Therefore, the length of the longest valid parentheses substring is 4.

For the second test case:

Given 'STR' ="))((", we can see that there is no valid substring present in the string. o, the answer will be 0.

Sample Input 2 :

	2 ()))((((())))))))(() **Sample Output 2 :** 10 2 Constraints: 1 <= T <= 5 1 <= |STR| <= 3000 where |STR| is the length of the given string.	

Hands-on No. : **4**

Topic : **Control Flow Statements, Arrays & Strings**

Date : **15/06/2026**

Solve the following problems

Question No.	Question Detail
1	A Strong Number is a number in which the sum of the factorial of each of its digits is equal to the number itself. For example, 145 is a strong number because: 1!+4!+5!=1+24+120=145 Write a Java program that checks whether a given number is a Strong Number or not. Sample Input :145 Sample Output :Strong Number Sample Input :141 Sample Output : Not a Strong Number
2	**Happy Number** **Problem statement:** Write an algorithm to determine if a number n is happy. A happy number is a number defined by the following process: • Starting with any positive integer, replace the number by the sum of the squares of its digits. • Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1. • Those numbers for which this process ends in 1 are happy. Return true if n is a happy number, and false if not. **Example 1:** **Input:** n = 19 **Output**: true **Explanation:** $1^2 + 9^2 = 82$ $8^2 + 2^2 = 68$ $6^2 + 8^2 = 100$ $1^2 + 0^2 + 0^2 = 1$ **Example 2:** **Input:** n = 2

	Output: false **Constraints:** 1 <= n <= 2^31 - 1
3	Given two strings, append them together (known as "concatenation") and return the result. However, if the concatenation creates a double-char, then omit one of the chars, so "abc" and "cat" yields "abcat". SampleInput: abc cat SampleOutput: abcat SampleInput: dog cat SampleOutput: dogcat SampleInput: abc SampleOutput: abc
4	Write a program to encrypt the text "INDIA" to "KPFKC" and decrypt "KPFKC" to get the original string "INDIA".
5	Given a square matrix **mat[][]**, turn it by 90 degrees in an anticlockwise direction without using any extra space **Examples:** **Input:** mat[][] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}} **Output: {{**3, 6, 9}, {2, 5, 8}, {1, 4, 7,}} **Input:** mat[][] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11,12} {13, 14, 15, 16}} **Output: {{**4, 8, 12, 16}, {3, 7, 11, 15}, {2, 6, 10, 14}, {1, 5, 9, 13}}
6	**Hollow rectangle Pattern** Write a program to generate and print a hollow rectangle pattern using asterisks (*). The user will specify the number of rows and columns, and the program will print a hollow rectangle of that size. **Explanation** **Dimensions:**

	• The pattern will have a specified number of rows and columns. • For example, if the user inputs 4 rows and 5 columns, the output will create a rectangle with a solid border made of asterisks and hollow spaces in between. **Pattern Structure:** • The first and last rows are filled with asterisks. • The rows in between have asterisks only at the start and end, with spaces in between. **Sample Input:** rows = 4; columns = 5 **Sample Output:** ***** * * * * *****
7	Find the Winner of the Circular Game Problem Statement: There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the ith friend brings you to the (i+1)th friend for 1 <= i < n, and moving clockwise from the nth friend brings you to the 1st friend. The rules of the game are as follows: 1. Start at the 1st friend. 2. Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once. 3. The last friend you counted leaves the circle and loses the game. 4. If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat. 5. Else, the last friend in the circle wins the game. Given the number of friends, n, and an integer k, return the winner of the game. Example 1: Input: n = 5, k = 2 Output: 3 Explanation: Here are the steps of the game: 1) Start at friend 1. 2) Count 2 friends clockwise, which are friends 1 and 2. 3) Friend 2 leaves the circle. Next start is friend 3. 4) Count 2 friends clockwise, which are friends 3 and 4. 5) Friend 4 leaves the circle. Next start is friend 5. 6) Count 2 friends clockwise, which are friends 5 and 1. 7) Friend 1 leaves the circle. Next start is friend 3. 8) Count 2 friends clockwise, which are friends 3 and 5. 9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.

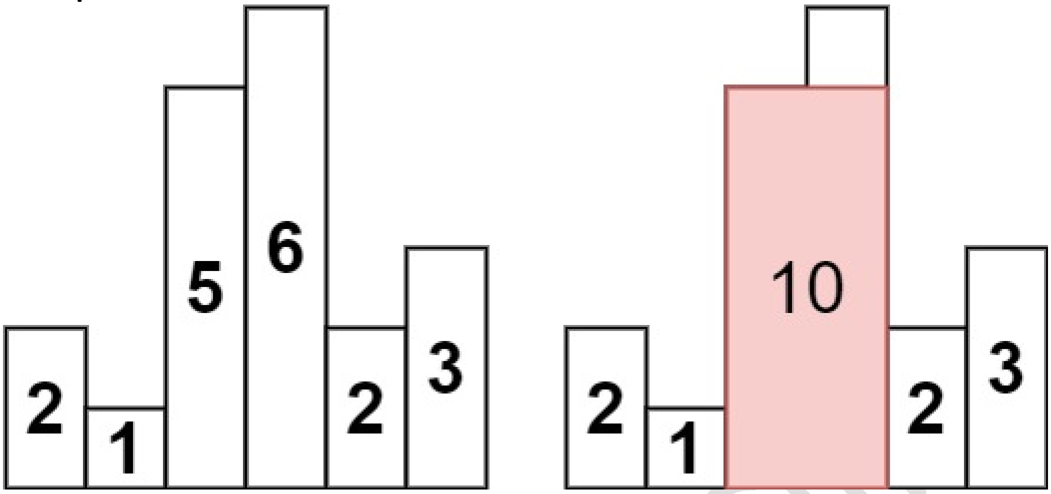
	Example 2: Input: n = 6, k = 5 Output: 1 Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1. Constraints: ● 1 <= k <= n <= 500
8	Ways To Express Problem statement : You are given the number ‘N’. The task is to find the number of ways to represent ‘N’ as a sum of two or more consecutive natural numbers. Example: N = 9 ‘9’ can be represented as: 2 + 3 + 4 = 9 4 + 5 = 9 The number of ways to represent ‘9’ as a sum of consecutive natural numbers is ‘2’. So, the answer is ‘2’. Note: 1. The numbers used to represent ‘N’ should be natural numbers (Integers greater than equal to 1). Sample Input 1: 2 21 15 Sample Output 1: 3 3 Explanation Of Sample Input 1: Test Case 1: ‘21’ can be represented as: 1 + 2 + 3 + 4 + 5 + 6 = 21 6 + 7 + 8 = 21 10 + 11 = 21 The number of ways to represent ‘21’ as a sum of consecutive natural numbers is ‘3’. So, the answer is ‘3’. Test Case 2: ‘15’ can be represented as: 1 + 2 + 3 + 4 + 5 = 15 4 + 5 + 6 = 15 7 + 8 = 15 The number of ways to represent ‘15’ as a sum of consecutive natural numbers is ‘3’. So, the answer is ‘3’. Sample Input 2: 2 18

	16 Sample Output 2: 2 0 Constraints: 1 <= T <= 1000 3 <= N <= 10 ^ 5 Where 'T' is the number of test cases, and 'N' is the given number.
9	Butterfly star pattern Problem statement: Create a program to print a butterfly star pattern consisting of stars in a symmetrical butterfly shape. The pattern should be printed such that the stars form wings on both sides of the centerline. The number of stars in each wing should decrease towards the centerline. Input 1 : rows = 5 Output 1: * * ** ** *** *** **** **** ********** **** **** *** *** ** ** * * Input 2: rows =7 Output 2 : * * ** ** *** *** **** **** ***** ***** ************ ************ ***** ***** **** **** *** *** ** ** * *
10	Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.

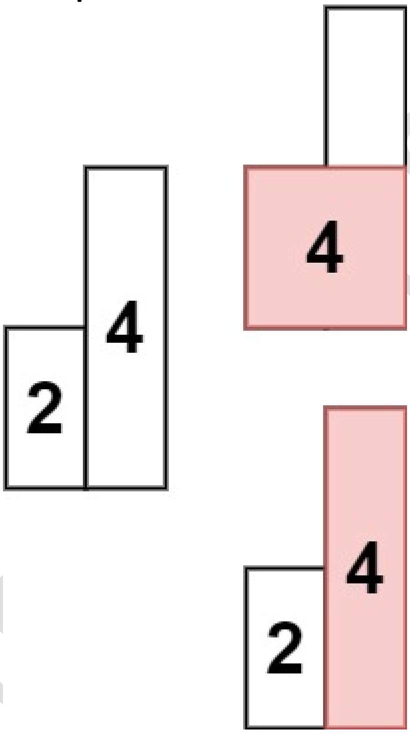
It is going to be hard but, hard does not mean impossible.

<table>
<tr>
<td></td>
<td>

Example 1:



Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.
Example 2:



Input: heights = [2,4]
Output: 4

Constraints:

- 1 <= heights.length <= 10^5
- 0 <= heights[i] <= 10^4

</td>
</tr>
<tr>
<td>11</td>
<td>

Hollow Left Triangle Pattern

</td>
</tr>
</table>

Write a program to print a hollow left triangle pattern, given the number of rows in a triangle(The hollow triangle star pattern is a triangle with hollow space in the middle.

Hint:

- **Outer Loop:** Iterate from 1 to the number of rows to control the current row.
- **Inner Loop:** Print * for the first column, last column, and the last row; otherwise, print spaces.
- **Line Break:** After finishing each row, move to the next line.

Sample Input: 6

Sample Output:

```
*
* *
*   *
*     *
*       *
* * * * * *
```

12

Longest Consecutive 1's

Problem Statement: Given a number N. Find the length of the longest consecutive 1s in its binary representation.

Example 1:

Input: N = 14

Output: 3

Explanation:

Binary representation of 14 is 1110, in which 111 is the longest consecutive set bits of length is 3.

Example 2:

Input: N = 222

Output: 4

Explanation:

Binary representation of 222 is 11011110, in which 1111 is the longest consecutive set bits of length 4.

Constraints:

1 <= N <= 10^6

13

Count Hills and Valleys in an Array

Problem statement : You are given a 0-indexed integer array nums. An index i is part of a hill in nums if the closest non-equal neighbors of i are smaller than nums[i].

Similarly, an index i is part of a valley in nums if the closest non-equal neighbors of i are larger than nums[i]. Adjacent indices i and j are part of the same hill or valley if nums[i] == nums[j].

Note that for an index to be part of a hill or valley, it must have a non-equal neighbor on both the left and right of the index.Return the number of hills and valleys in nums.

Example 1:

Input: nums = [2,4,1,1,6,5]

Output: 3

Explanation:

At index 0: There is no non-equal neighbor of 2 on the left, so index 0 is neither a hill nor a valley.

At index 1: The closest non-equal neighbors of 4 are 2 and 1. Since 4 > 2 and 4 > 1, index 1 is a hill.

At index 2: The closest non-equal neighbors of 1 are 4 and 6. Since 1 < 4 and 1 < 6, index 2 is a valley.

At index 3: The closest non-equal neighbors of 1 are 4 and 6. Since 1 < 4 and 1 < 6, index 3 is a valley, but note that it is part of the same valley as index 2.

At index 4: The closest non-equal neighbors of 6 are 1 and 5. Since 6 > 1 and 6 > 5, index 4 is a hill.

At index 5: There is no non-equal neighbor of 5 on the right, so index 5 is neither a hill nor a valley.

There are 3 hills and valleys so we return 3.

Example 2:

Input: nums = [6,6,5,5,4,1]

Output: 0

Explanation:

At index 0: There is no non-equal neighbor of 6 on the left, so index 0 is neither a hill nor a valley.

At index 1: There is no non-equal neighbor of 6 on the left, so index 1 is neither a hill nor a valley.

At index 2: The closest non-equal neighbors of 5 are 6 and 4. Since 5 < 6 and 5 > 4, index 2 is neither a hill nor a valley.

At index 3: The closest non-equal neighbors of 5 are 6 and 4. Since 5 < 6 and 5 > 4, index 3 is neither a hill nor a valley.

At index 4: The closest non-equal neighbors of 4 are 5 and 1. Since 4 < 5 and 4 > 1, index 4 is neither a hill nor a valley.

	At index 5: There is no non-equal neighbor of 1 on the right, so index 5 is neither a hill nor a valley. There are 0 hills and valleys so we return 0. **Constraints:** 3 <= nums.length <= 100 1 <= nums[i] <= 100
14	**Reverse Bits of a Number** Given a 32-bit unsigned integer, **reverse its bits** and return the resulting number. **Sample Input 1** **n = 43261596** **Sample Output 1** 964176192 **Sample Input 2** **n = 1** **Sample Output 2** **2147483648**
15	**Minimum Bit Flips to Convert Number** **Problem Statement:** A bit flip of a number x is choosing a bit in the binary representation of x and flipping it from either 0 to 1 or 1 to 0. Given two integers start and goal, return the minimum number of bit flips to convert start to goal. **Example 1:** **Input:** start = 10, goal = 7 **Output:** 3 **Explanation:** The binary representation of 10 and 7 are 1010 and 0111 respectively. We can convert 10 to 7 in 3 steps: - Flip the first bit from the right: 1010 -> 1011. - Flip the third bit from the right: 1011 -> 1111. - Flip the fourth bit from the right: 1111 -> 0111. It can be shown we cannot convert 10 to 7 in less than 3 steps. Hence, we return 3. **Example 2:** **Input:** start = 3, goal = 4 **Output:** 3 **Explanation:** The binary representation of 3 and 4 are 011 and 100 respectively. We can convert 3 to 4 in 3 steps: - Flip the first bit from the right: 011 -> 010. - Flip the second bit from the right: 010 -> 000. - Flip the third bit from the right: 000 -> 100.

	It can be shown we cannot convert 3 to 4 in less than 3 steps. Hence, we return 3. **Constraints:** 0 <= start, goal <= 10^9
	K – star Pattern Write a program to print the below pattern using stars, given the number of rows in each half of the pattern(This is a combination of patterns discussed earlier) **Sample Input:** 5 (rows) **Sample Output:** *
16	**Move Zeros to end** **Problem statement :** Given an unsorted array of integers, you have to move the array elements in a way such that all the zeroes are transferred to the end, and all the non-zero elements are moved to the front. The non-zero elements must be ordered in their order of appearance. For example, if the input array is: [0, 1, -2, 3, 4, 0, 5, -27, 9, 0], then the output array must be: [1, -2, 3, 4, 5, -27, 9, 0, 0, 0]. Expected Complexity: Try doing it in O(n) time complexity and O(1) space complexity. Here, 'n' is the size of the array. **Sample Input 1:** 1 7 2 0 4 1 3 0 28 **Sample Output 1**: 2 4 1 3 28 0 0 The explanation for Sample Output 1 :All the zeros are moved towards the end of the array, and the non-zero elements are pushed towards the left, maintaining their

	order with respect to the original array. **Sample Input 2**: 1 5 0 3 0 2 0 **Sample Output 2**: 3 2 0 0 0
17	**Count ways to reach the n'th stair** **Problem Statement** : There are n stairs, a person standing at the bottom wants to reach the top. The person can climb either 1 stair or 2 stairs at a time. Count the number of ways, the person can reach the top (order does matter). **Example 1:** **Input:** n = 4 **Output:** 5 **Explanation:** You can reach 4th stair in 5 ways. Way 1: Climb 2 stairs at a time. Way 2: Climb 1 stair at a time. Way 3: Climb 2 stairs, then 1 stair and then 1 stair. Way 4: Climb 1 stair, then 2 stairs then 1 stair. Way 5: Climb 1 stair, then 1 stair and then 2 stairs. **Example 2:** **Input:** n = 10 **Output**: 89 **Explanation:** There are 89 ways to reach the 10th stair.

	Constraints: 1 ≤ n ≤ 10^4
18	**Max Consecutive Ones** **Problem Statement :** Given a binary array arr[] (containing only 0s and 1s) and an integer k, you are allowed to flip at most k 0s to 1s. Find the maximum number of consecutive 1's that can be obtained in the array after performing the operation at most k times. **Examples:** Input: arr[] = [1, 0, 1], k = 1 Output: 3 Explanation: Maximum subarray of consecutive 1's is of size 3 which can be obtained after flipping the zero present at the 1st index.Input: arr[] = [1, 0, 0, 1, 0, 1, 0, 1], k = 2 Output: 5 Explanation: By flipping the zeroes at indices 4 and 6, we get the longest subarray from index 3 to 7 containing all 1’s. Input: arr[] = [1, 1], k = 2 Output: 2 Explanation: Since the array is already having the max consecutive 1's, hence we dont need to perform any operation. Hence the answer is 2 **Constraints**: 1 <= arr.size() <= 10^5 0 <= k <= arr.size() 0 <= arr[i] <= 1

Hands-on No. : **5**

Topic : **Collections, Stack and Queue**

Date : **16/06/2026**

Solve the following problems

Question No.	Question Detail
1	**Remove All Occurrences of a Given Element** **Problem Statement:** Remove all occurrences of a specific element from a list. **Sample Input:** List: [1, 2, 3, 2, 4] **Element: 2** **Sample Output:** [1, 3, 4] **Hint:** Use Iterator to safely remove while iterating.
2	**Sort a Map by Values** **Problem Statement:** Sort a HashMap by its values in descending order. **Sample Input:** {A=5, B=2, C=8} **Sample Output:** {C=8, A=5, B=2} **Hint:** Convert entrySet() to a list, sort, and then collect into LinkedHashMap.
3	**Merge Two Maps** **Problem Statement:** Merge two maps. If keys are the same, sum their values. **Sample Input:** Map1: {A=5, B=3} Map2: {A=2, C=4} **Sample Output:** {A=7, B=3, C=4}

It is going to be hard but, hard does not mean impossible.

4	**Find First Non-Repeating Element** **Problem Statement:** Find the first element in a list that does not repeat. **Sample Input:** [4, 5, 1, 2, 0, 4] **Sample Output:** 5
5	**Flatten a List of Lists** **Problem Statement:** Convert List<List<Integer>> into List<Integer>. **Sample Input:** [[1, 2], [3, 4]] **Sample Output:** [1, 2, 3, 4]
6	**Sort List of Objects by Field** **Problem Statement:** Sort a list of objects by one of their attributes. **Sample Input:** Person{name="Alice", age=25}, Person{name="Bob", age=20} **Sample Output (sorted by age):** Bob(20), Alice(25)
7	**Find the Difference of Two Arrays** **Problem statement :** Given two **0-indexed** integer arrays nums1 and nums2, return a list answer of size 2 where: • answer[0] is a list of all **distinct** integers in nums1 which are **not** present in nums2. • answer[1] is a list of all **distinct** integers in nums2 which are **not** present in nums1. **Note** that the integers in the lists may be returned in **any** order. **Example 1:** **Input:** nums1 = [1,2,3], nums2 = [2,4,6] **Output:** [[1,3],[4,6]] **Explanation:**

	For nums1, nums1[1] = 2 is present at index 0 of nums2, whereas nums1[0] = 1 and nums1[2] = 3 are not present in nums2. Therefore, answer[0] = [1,3]. For nums2, nums2[0] = 2 is present at index 1 of nums1, whereas nums2[1] = 4 and nums2[2] = 6 are not present in nums2. Therefore, answer[1] = [4,6]. **Example 2:** **Input:** nums1 = [1,2,3,3], nums2 = [1,1,2,2] **Output:** [[3],[]] **Explanation:** For nums1, nums1[2] and nums1[3] are not present in nums2. Since nums1[2] == nums1[3], their value is only included once and answer[0] = [3]. Every integer in nums2 is present in nums1. Therefore, answer[1] = []. **Constraints:** • 1 <= nums1.length, nums2.length <= 1000 • -1000 <= nums1[i], nums2[i] <= 1000
8	**Check if a String Is an Acronym of Words** **Problem statement:** Given an array of strings words and a string s, determine if s is an acronym of words. The string s is considered an acronym of words if it can be formed by concatenating the first character of each string in words in order. For example, "ab" can be formed from ["apple", "banana"], but it can't be formed from ["bear", "aardvark"]. Return true if s is an acronym of words, and false otherwise. **Example 1:** **Input:** words = ["alice","bob","charlie"], s = "abc" **Output**: true **Explanation:** The first character in the words "alice", "bob", and "charlie" are 'a', 'b', and 'c', respectively. Hence, s = "abc" is the acronym. **Example 2:** **Input:** words = ["an","apple"], s = "a" **Output:** false

	Explanation: The first character in the words "an" and "apple" are 'a' and 'a', respectively. The acronym formed by concatenating these characters is "aa". Hence, s = "a" is not the acronym. **Example 3:** **Input**: words = ["never","gonna","give","up","on","you"], s = "ngguoy" **Output**: true **Explanation**: By concatenating the first character of the words in the array, we get the string "ngguoy". Hence, s = "ngguoy" is the acronym. **Constraints:** 1 <= words.length <= 100 1 <= words[i].length <= 10 1 <= s.length <= 100 words[i] and s consist of lowercase English letters.
9	**Find Words That Can Be Formed by Characters** **Problem statement :** You are given an array of strings words and a string chars. A string is good if it can be formed by characters from chars (each character can only be used once). Return the sum of lengths of all good strings in words. **Example 1:** **Input**: words = ["cat","bt","hat","tree"], chars = "atach" **Output:** 6 **Explanation**: The strings that can be formed are "cat" and "hat" so the answer is 3 + 3 = 6. **Example 2:** **Input**: words = ["hello","world","leetcode"], chars = "welldonehoneyr" **Output:** 10 **Explanation:** The strings that can be formed are "hello" and "world" so the answer is 5 + 5 = 10. **Constraints:** 1 <= words.length <= 1000

	1 <= words[i].length, chars.length <= 100 words[i] and chars consist of lowercase English letters.
10	**Redistribute Characters to Make All Strings Equal** **Problem statement :** You are given an array of strings words (0-indexed). In one operation, pick two distinct indices i and j, where words[i] is a non-empty string, and move any character from words[i] to any position in words[j]. Return true if you can make every string in words equal using any number of operations, and false otherwise. **Example 1:** **Input:** words = ["abc","aabc","bc"] **Output:** true **Explanation:** Move the first 'a' in words[1] to the front of words[2], to make words[1] = "abc" and words[2] = "abc". All the strings are now equal to "abc", so return true. **Example 2:** **Input:** words = ["ab","a"] **Output:** false **Explanation**: It is impossible to make all the strings equal using the operation. **Constraints:** 1 <= words.length <= 100 1 <= words[i].length <= 100 words[i] consists of lowercase English letters.
11	**First Unique Character in a String** **Problem statement :** Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1. **Example 1:** **Input:** s = "leetcode" **Output**: 0 **Example 2:** **Input:** s = "loveleetcode" **Output**: 2 **Example 3:** **Input**: s = "aabb"

	Output: -1 **Constraints:** 1 <= s.length <= 10^5 s consists of only lowercase English letters.

12	**Kth Largest Element in a Stream**
	
	Problem statement : Design a class to find the kth largest element in a stream. Note that it is the kth largest element in the sorted order, not the kth distinct element.
	Implement KthLargest class:
	KthLargest(int k, int[] nums) Initializes the object with the integer k and the stream of integers nums.
	int add(int val) Appends the integer val to the stream and returns the element representing the kth largest element in the stream.
	
	Example 1:
	Input
	["KthLargest", "add", "add", "add", "add", "add"]
	[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]
	Output
	[null, 4, 5, 5, 8, 8]
	
	Explanation
	KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);
	kthLargest.add(3); // return 4
	kthLargest.add(5); // return 5
	kthLargest.add(10); // return 5
	kthLargest.add(9); // return 8
	kthLargest.add(4); // return 8
	
	Constraints:
	1 <= k <= 10^4
	0 <= nums.length <= 10^4
	-10^4 <= nums[i] <= 10^4
	-10^4 <= val <= 10^4
	At most 10^4 calls will be made to add.

<table>
<tr><td></td><td>It is guaranteed that there will be at least k elements in the array when you search for the kth element.</td></tr>
<tr><td>13</td><td>Top K Frequent Elements

Problem Statement: Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.

Example 1:
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]

Example 2:
Input: nums = [1], k = 1
Output: [1]

Constraints:

1 <= nums.length <= 10^5
-10^4 <= nums[i] <= 10^4
k is in the range [1, the number of unique elements in the array].
It is guaranteed that the answer is unique.</td></tr>
<tr><td>14</td><td>Find Smallest Letter Greater Than Target

Problem Statement : You are given an array of characters letters that is sorted in non-decreasing order, and a character target. There are at least two different characters in letters.

Return the smallest character in letters that is lexicographically greater than target. If such a character does not exist, return the first character in letters.

Example 1:

Input: letters = ["c","f","j"], target = "a"
Output: "c"
Explanation: The smallest character that is lexicographically greater than 'a' in letters is 'c'.

Example 2:

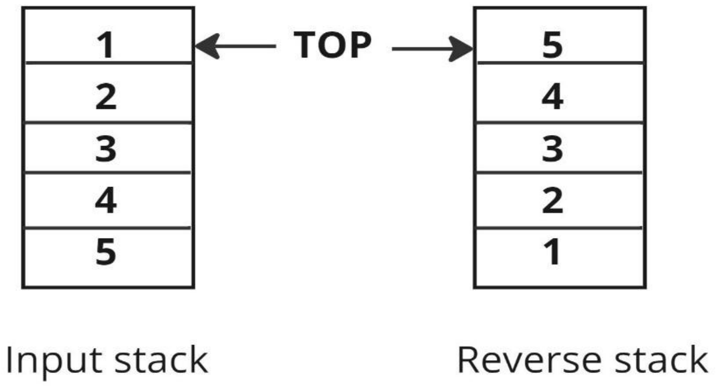
Input: letters = ["c","f","j"], target = "c"
Output: "f"
Explanation: The smallest character that is lexicographically greater than 'c' in letters is 'f'.

Example 3:

Input: letters = ["x","x","y","y"], target = "z"
Output: "x"</td></tr>
</table>

	Explanation: There are no characters in letters that is lexicographically greater than 'z' so we return letters[0]. **Constraints:** 2 <= letters.length <= 10^4 letters[i] is a lowercase English letter. letters is sorted in non-decreasing order. letters contains at least two different characters. target is a lowercase English letter.
15	**Find Transition Point** **Problem Statement :** Given a sorted array containing only 0s and 1s, find the transition point, i.e., the first index where 1 was observed, and before that, only 0 was observed. **Example 1:** **Input:** N = 5 arr[] = {0,0,0,1,1} **Output:** 3 **Explanation:** index 3 is the transition point where 1 begins. **Example 2:** **Input:** N = 4 arr[] = {0,0,0,0} **Output:** -1 **Explanation:** Since, there is no "1", the answer is -1. **Constraints:** 1 ≤ N ≤ 10^5 0 ≤ arr[i] ≤ 1
16	**Array Subset of another array** **Problem Statement :** Given two arrays: a1[0..n-1] of size n and a2[0..m-1] of size m, where both arrays may contain duplicate elements. The task is to determine

	whether array a2 is a subset of array a1. It's important to note that both arrays can be sorted or unsorted. Additionally, each occurrence of a duplicate element within an array is considered as a separate element of the set. **Example 1:** **Input:** a1[] = {11, 7, 1, 13, 21, 3, 7, 3} a2[] = {11, 3, 7, 1, 7} **Output:** Yes **Explanation:** a2[] is a subset of a1[] **Example 2:** **Input:** a1[] = {1, 2, 3, 4, 4, 5, 6} a2[] = {1, 2, 4} **Output:** Yes **Explanation:** a2[] is a subset of a1[] **Example 3:** **Input:** a1[] = {10, 5, 2, 23, 19} a2[] = {19, 5, 3} **Output:** No **Explanation:** a2[] is not a subset of a1[] **Constraints:** 1 <= n,m <= 10^5 1 <= a1[i], a2[j] <= 10^6
17	**Reverse Stack Using Recursion** **Problem statement**

	Reverse a given stack of 'N' integers using recursion. You are required to make changes in the input parameter itself. Note: You are not allowed to use any extra space other than the internal stack space used due to recursion. **Example**: Input: [1,2,3,4,5] Output: [5,4,3,2,1]  Sample Input 1 : 3 2 1 3 Sample Output 1 : 3 1 2 Explanation to Sample Input 1 : Reverse of a give stack is 3 1 2 where first element becomes last and last becomes first. Sample Input 2 : 2 3 2 Sample Output 2 : 2 3 **Constraints** : 0 <= N <= 10^3 Where 'N' is the number of elements in the given stack.
18	**Reverse Substrings Between Each Pair of Parentheses**

	Problem Statement: You are given a string s that consists of lower case English letters and brackets. Reverse the strings in each pair of matching parentheses, starting from the innermost one.Your result should not contain any brackets. **Example 1:** Input: s = "(abcd)" Output: "dcba" **Example 2:** Input: s = "(u(love)i)" Output: "iloveu" Explanation: The substring "love" is reversed first, then the whole string is reversed. **Example 3:** Input: s = "(ed(et(oc))el)" Output: "leetcode" Explanation: First, we reverse the substring "oc", then "etco", and finally, the whole string. **Constraints:** 1 <= s.length <= 2000 s only contains lower case English characters and parentheses. It is guaranteed that all parentheses are balanced.

Hands-on No. : **7**

Topic : **Linked List, Stack and Queue**

Date : **17/06/2026**

Solve the following problems

Question No.	Question Detail as
1	**Remove duplicates from a sorted Doubly Linked List** Problem statement : A doubly-linked list is a data structure that consists of sequentially linked nodes, and the nodes have reference to both the previous and the next nodes in the sequence of nodes. You are given a sorted doubly linked list of size 'n'. Remove all the duplicate nodes present in the linked list. **Example 1** : Input: Linked List: 1 <-> 2 <-> 2 <-> 2 <-> 3 Output: Modified Linked List: 1 <-> 2 <-> 3 Explanation: We will delete the duplicate values '2' present in the linked list. Sample Input 1 : 5 1 2 2 2 3 Sample Output 1 : 1 2 3 Explanation For Sample Input 1 : We will delete the duplicate values '2' present in the linked list. **Example 2** : Sample Input 2 : 4 1 2 3 4 Sample Output 2 : 1 2 3 4

	Explanation For Sample Input 2 : The list contains no duplicates, so the final list is unchanged. Constraints : 1 <= 'n' <= 10^5 1 <= 'data' in any node <= 10^6
2	**Infix To Postfix** **Problem statement :** You are given a string 'exp' which is a valid infix expression. Convert the given infix expression to postfix expression. **Note:** Infix notation is a method of writing mathematical expressions in which operators are placed between operands. For example, "3 + 4" represents the addition of 3 and 4. Postfix notation is a method of writing mathematical expressions in which operators are placed after the operands. For example, "3 4 +" represents the addition of 3 and 4. Expression contains digits, lower case English letters, '(', ')', '+', '-', '*', '/', '^'. **Example:** **Input:** exp = '3+4*8' **Output:** 348*+ **Explanation:** Here multiplication is performed first and then the addition operation. Hence postfix expression is 3 4 8 * +. **Sample Input 1:** 3^(1+1) **Expected Answer:** 311+^ **Output printed on console:** 311+^ **Explanation of Sample Input 1:** For this testcase, we will evaluate 'b' = (1+1) first. Hence it's equivalent postfix expression will be "11+".

	Next we will evaluate 3^b. It's equivalent postfix expression will be "3b^". Replacing 'b' with it's equivalent postfix we get "311+^". **Sample Input 2:** a+b+c+d-e **Expected Answer**: ab+c+d+e- **Output printed on console:** ab+c+d+e- **Constraints:** 1 <= 'n' <= 5000 'n', is the length of EXP The expression contains digits, lower case English letters, '(', ')', '+', '-', '*', '/', '^'.
3	**Arithmetic Expression Evaluation** **Problem statement** You are given a string 'expression' consists of characters '+', '-', '*', '/', '(', ')' and '0' to '9', that represents an Arithmetic Expression in Infix Notation. Your task is to evaluate this Arithmetic Expression. In Infix Notation, operators are written in-between their operands. **Note :** 1. We consider the '/' operator as the floor division. 2. Operators '*' and '/' expression has higher precedence over operators'+' and '-' 3. String expression always starts with '(' and ends with ')'. 4. It is guaranteed that 'expression' represents' a valid expression in Infix notation. 5. It is guaranteed that there will be no case that requires division by 0. 6. No characters other than those mentioned above are present in the string.

	7. It is guaranteed that the operands and final result will fit in a 32-bit integer. For example : Consider string 'expression' = '((2+3)*(5/2))'. Then it's value after evaluation will be ((5)*(2)) = 10. **Sample Input 1 :** 2 (2) ((2+3)*(5/2)) **Sample Output 1 :** 2 10 **Explanation For Sample Input 1 :** **Test case 1 :** The value of the expression "(2)" will be 2 after evaluation. **Test case 2 :** See the problem statement for an explanation. **Sample Input 2 :** 2 (121+(101+0)) (3*(5+2)*(10-7)) **Sample Output 2 :** 222 63 **Constraints :** 1 <= T <= 50 3 <= |expression| <= 10^4
4	**Postfix To Prefix** **Problem Statement:** You are given a string denoting a valid Postfix expression containing '+', '-', '*', '/' and lowercase letters. Convert the given Postfix expression into a Prefix expression.

	Note: Postfix notation is a method of writing mathematical expressions in which operators are placed after the operands. For example, "a b +" represents the addition of a and b. Prefix notation is a method of writing mathematical expressions in which operators are placed before the operands. For example, "+ a b" represents the addition of a and b. Expression contains lowercase English letters, '+', '-', '*', and '/'. **Example:** Input: abc*+ Output: +a*bc Explanation:For the given postfix expression, infix expression is a+b*c. And it's corresponding prefix expression is +a*bc. **Sample Input 1:** ab+cd-* Expected Answer: *+ab-cd Output on console: *+ab-cd Explanation Of Sample Input 1: For this testcase, the infix expression will be (a + b) * (c - d). Hence, our prefix expression will be *+ab-cd. **Sample Input 2:** ab+c- Expected Answer: -+abc Output on console: -+abc **Constraints:** 1 <= |s| <= 10^5
5	**Remove All Adjacent Duplicates in String**

		Problem Statement: You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them. We repeatedly make duplicate removals on s until we no longer can. Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique. **Example 1:** **Input:** s = "abbaca" **Output:** "ca" **Explanation:** For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca". **Example 2:** **Input:** s = "azxxzy" **Output:** "ay" **Constraints:** 1 <= s.length <= 10^5 s consists of lowercase English letters.

Hands-on No. **:1**

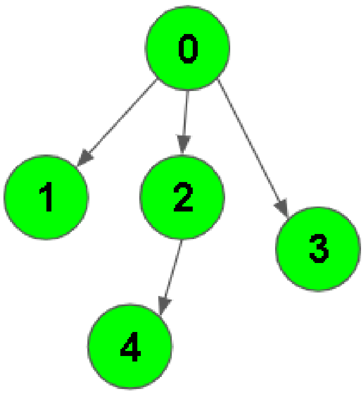
Topic **: Graph**

Date **: 19-06-26**

Solve the following problems

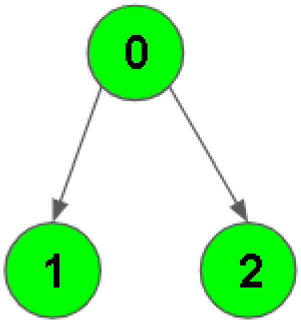
<table>
<tr><th>Q. No.</th><th>Question Detail</th><th>Level</th></tr>
<tr><td>1</td><td>BFS of Graph
Problem Statement: Given a directed graph. The task is to do Breadth First Traversal of this graph starting from 0.

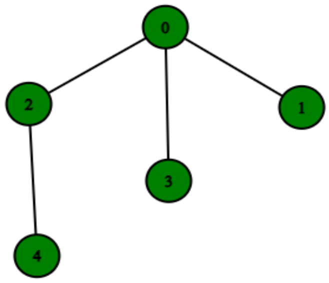
Note: One can move from node u to node v only if there's an edge from u to v. Find the BFS traversal of the graph starting from the 0th vertex, from left to right according to the input graph. Also, you should only take nodes directly or indirectly connected from Node 0 in consideration.

Example 1:
Input:
V = 5, E = 4
adj = {{1,2,3},{},{4},{},{}}


Output:
0 1 2 3 4</td><td>Basic</td></tr>
</table>

It is going to be hard but, hard does not mean impossible.

	Explanation: 0 is connected to 1 , 2 , 3. 2 is connected to 4. so starting from 0, it will go to 1 then 2 then 3. After this 2 to 4, thus bfs will be 0 1 2 3 4. **Example 2:** **Input:** V = 3, E = 2 adj = {{1,2},{},{}}  **Output:** 0 1 2 **Explanation:** 0 is connected to 1 , 2. so starting from 0, it will go to 1 then 2, thus bfs will be 0 1 2. **Constraints:** 1 ≤ V, E ≤ 10^4	
2	**DFS of Graph** **Problem Statement :** You are given a connected undirected graph. Perform a Depth First Traversal of the graph. Note: Use the recursive approach to find the DFS traversal of the graph starting from the 0th vertex from left to right according to the graph. **Example 1:** **Input:** V = 5 , adj = [[2,3,1] , [0], [0,4], [0], [2]]	Basic



Output: 0 2 4 3 1

Explanation:

0 is connected to 2, 3, 1.

1 is connected to 0.

2 is connected to 0 and 4.

3 is connected to 0.

4 is connected to 2.

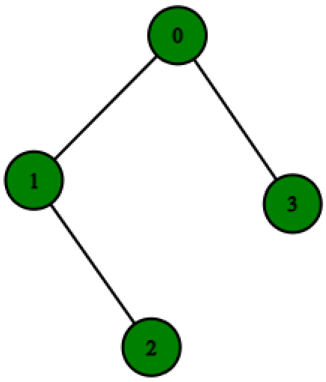
so starting from 0, it will go to 2 then 4,

and then 3 and 1.

Thus dfs will be 0 2 4 3 1.

Example 2:

Input: V = 4, adj = [[1,3], [2,0], [1], [0]]



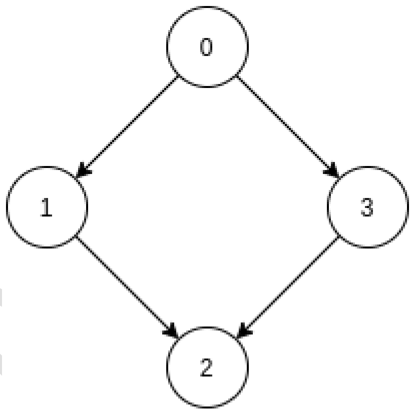
Output: 0 1 2 3

Explanation:

0 is connected to 1 , 3.

1 is connected to 0, 2.

2 is connected to 1.

	3 is connected to 0. so starting from 0, it will go to 1 then 2 then back to 0 then 0 to 3 thus dfs will be 0 1 2 3. **Constraints:** 1 ≤ V, E ≤ 10^4	
3	**Topological Sort** **Problem statement** A Directed Acyclic Graph (DAG) is a directed graph that contains no cycles. *Topological Sorting of DAG is a linear ordering of vertices such that for every directed edge from vertex 'u' to vertex 'v', vertex 'u' comes before 'v' in the ordering. Topological Sorting for a graph is not possible if the graph is not a DAG.* Given a DAG consisting of 'V' vertices and 'E' edges, you need to find out any topological sorting of this DAG. Return an array of size 'V' representing the topological sort of the vertices of the given DAG. For example, Consider the DAG shown in the picture.  In this graph, there are directed edges from 0 to 1 and 0 to 3, so 0 should come before 1 and 3. Also, there are directed edges from 1 to 2 and 3 to 2 so 1 and 3 should come before 2. So, The topological sorting of this DAG is {0 1 3 2}. Note that there are multiple topological sortings possible for a DAG. For the graph given above one another topological sorting is: {0, 3, 1, 2} Note:	Easy

1. It is guaranteed that the given graph is DAG.
2. There will be no multiple edges and self-loops in the given DAG.
3. There can be multiple correct solutions, you can find any one of them.
4. Don't print anything, just return an array representing the topological sort of the vertices of the given DAG.

Sample Input 1:

2
2 1
1 0
3 2
0 1
0 2

Sample Output 1:

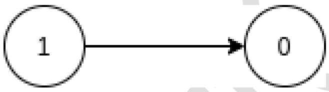
1 0
0 2 1

Explanation of Sample Input 1:

Test case 1:

The number of vertices 'V' = 2 and number of edges 'E' = 1.

The graph is shown in the picture:

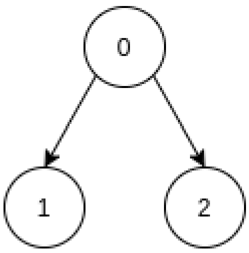


The topological sorting of this graph should be {1, 0} as there is a directed edge from vertex 1 to vertex 0, thus 1 should come before 0 according to the given definition of topological sorting.

Test case 2:

The number of vertices 'V' = 3 and number of edges 'E' = 2.

The graph is shown in the picture:



	As there are two directed edges starting from 0, so 0 should come before 1 and 2 in topological sorting. Thus the topological sorting of this graph should be {0, 2, 1} or {0, 1, 2} **Sample Input 2:** 2 1 0 4 4 0 1 0 3 1 2 3 2 **Sample Output 2:** 0 0 1 3 2 **Explanation of Sample Input 2:** Test case 1: There is only a single vertex in the graph that is 0, so its topological sort will be {0}. Test case 2: See problem statement for its explanation **Constraints:** 1 <= T <= 50 1 <= V <= 10^4 0 <= E <= 10^4 0 <= u, v < V Where ‘T’ is the total number of test cases, ‘V’ is the number of vertices, ‘E’ is the number of edges, and ‘u’ and ‘v’ both represent the vertex of a given graph.	
4	**Dijkstra's shortest path** **Problem statement** You have been given an undirected, connected graph of ‘V’ vertices (labelled from 0 to V-1) and ‘E’ edges. Each edge connecting two nodes 'u' and 'v' has a weight denoting the distance between them.	Easy

Your task is to find the shortest path distance from the source node 'S' to all the vertices. You have to return a list of integers denoting the shortest distance between each vertex and source vertex 'S'.

Note:

1. There are no self-loops(an edge connecting the vertex to itself) in the given graph.

2. There are no parallel edges i.e no two vertices are directly connected by more than 1 edge.

For Example:

Input:

4 5
0 1 5
0 2 8
1 2 9
1 3 2
2 3 6

The source node is node 0.

The shortest distance from node 0 to node 0 is 0.

The shortest distance from node 0 to node 1 is 5. In the above figure, the green path represents this distance. The path goes from node 0->1, giving distance = 5.

The shortest distance from node 0 to node 2 is 8. In the above figure, the pink path represents this distance. The path goes from node 0->2, giving distance = 8.

The shortest distance from node 0 to node 3 is 7. In the above figure, the yellow path represents this distance. The path goes from node 0->1->3, giving distance = 7.

Sample Input 1

5 7 0

0 1 7

0 2 1

0 3 2

1 2 3

1 3 5

1 4 1

3 4 7

Sample Output 1

0 4 1 2 5

Explanation of sample input 1

The source node is node 0.

The shortest distance from node 0 to node 0 is 0.

The shortest distance from node 0 to node 1 is 4. In the above figure, the green path represents this distance. The path goes from node 0->2->1, giving distance = 1+3 = 4.

The shortest distance from node 0 to node 2 is 1. In the above figure, the red path represents this distance. The path goes from node 0->2, giving distance = 1

The shortest distance from node 0 to node 3 is 2. In the above figure, the pink path represents this distance. The path goes from node 0->3, giving distance = 2.

The shortest distance from node 0 to node 4 is 5. In the above figure, the yellow path represents this distance. The path goes from node 0->2->1->4, giving distance = 1+3+1 = 5.

Sample Input 2:

3 3 2

1 0 9

2 1 8

0 2 4

	Sample Output 2: 4 8 0 **Constraints:** 1 <= 'V' <= 10^5 1 <= 'E' <= 10^5 1 <= distance between vertices <= 10^5	
5	**Prims Algorithm** **Problem Statement :** Given an undirected graph represented with edges, implement Prim's Algorithm to find the Minimum Spanning Tree (MST) of the graph. The graph has the following edges: • Edge from vertex 0 to vertex 1 with weight 1 • Edge from vertex 1 to vertex 2 with weight 2 • Edge from vertex 2 to vertex 3 with weight 3 • Edge from vertex 0 to vertex 3 with weight 4 Print the edges included in the MST along with their corresponding weights. **Output:** Edge 0-1 with weight 1 Edge 1-2 with weight 2 Edge 2-3 with weight 3	Easy
6	**Kruskal's Algorithm** **Problem Statement** Given an undirected graph with vertices and edges, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the graph. The graph is represented as an array of edges, where each edge has a source vertex, a destination vertex, and a weight. **Input graph:** • The graph has 4 vertices and 5 edges. • The edges of the graph are defined as follows: ○ Edge from vertex 0 to vertex 1 with weight 7 ○ Edge from vertex 0 to vertex 2 with weight 6 ○ Edge from vertex 0 to vertex 3 with weight 5	Easy

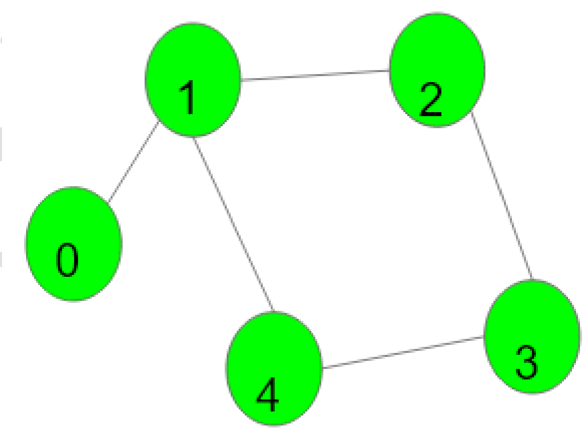
	○ Edge from vertex 1 to vertex 3 with weight 15 ○ Edge from vertex 2 to vertex 3 with weight 2 **Output** : Edges in MST: 2 - 3, weight = 2 0 - 3, weight = 5 0 - 1, weight = 7 Total Cost of MST: 14	

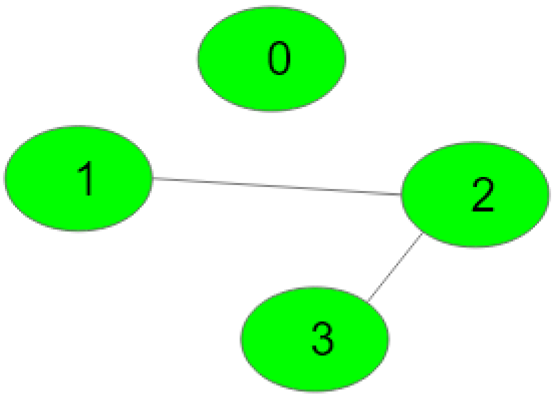
Hands-on No. : **1**

Topic : **Graph**

Date : **19-06-26**

Solve the following problems

Q. No.	Question Detail	Level
1	**Undirected Graph Cycle** Given an undirected graph with V vertices labelled from 0 to V-1 and E edges, check whether it contains any cycle or not. Graph is in the form of adjacency list where adj[i] contains all the nodes ith node is having edge with. Example 1: Input: V = 5, E = 5 adj = {{1}, {0, 2, 4}, {1, 3}, {2, 4}, {1, 3}} Output: 1 Explanation:  1->2->3->4->1 is a cycle. Example 2:	Easy

	Input: V = 4, E = 2 adj = {{}, {2}, {1, 3}, {2}} Output: 0 Explanation:  No cycle in the graph. **Constraints**: 1 ≤ V, E ≤ 10^5	
2	**Detect Cycle in a Directed Graph** **Problem statement** Given a directed graph, check whether the graph contains a cycle or not. Your function should return true if the given graph contains at least one cycle, else return false. **Sample Input 1:** 2 4 4 0 1 1 2 2 3 3 0 3 3 1 0 1 2 0 2 **Sample Output 1:**	Basic

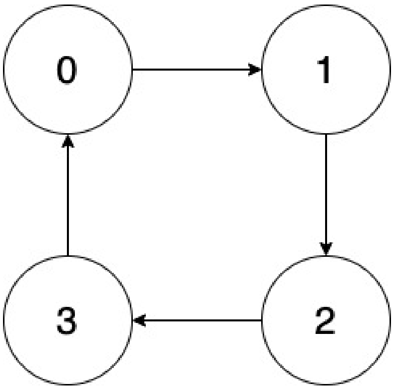
true

false

Explanation for Sample Input 1:

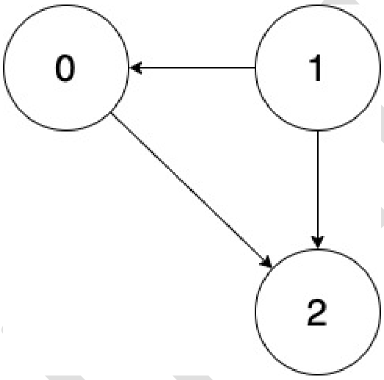
In the first case,

From node 0 we can reach 0 again by following this sequence of nodes in the path: 0,1,2,3,0. As we can see in the image below.



In the second case,

There is no cycle in the given graph. As we can see in the image below.



Sample Input 2:

2

3 2

0 1

0 2

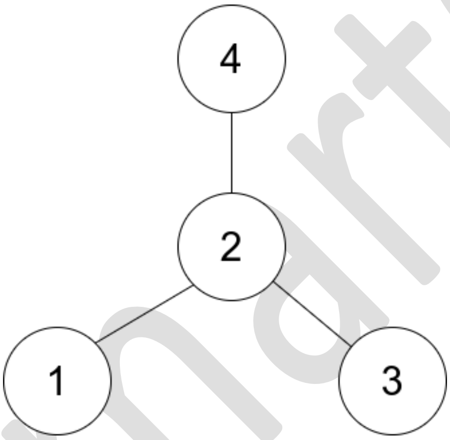
3 3

1 1

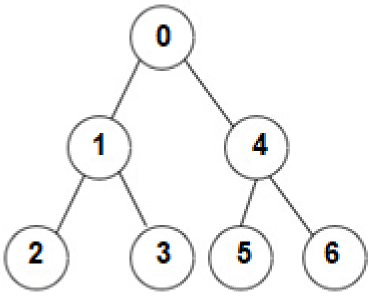
0 2

1 2

Sample Output 2:

	false true **Constraints:** 1 <= T <= 10 1 <= V <= 10^3 0 <= E <= 10^3 0 <= A, B < V	
3	**Find Center of Star Graph** **Problem Statement:** There is an undirected star graph consisting of n nodes labeled from 1 to n. A star graph is a graph where there is one center node and exactly n - 1 edges that connect the center node with every other node. You are given a 2D integer array edges where each edges[i] = [ui, vi] indicates that there is an edge between the nodes ui and vi. Return the center of the given star graph. **Example 1:**  Input: edges = [[1,2],[2,3],[4,2]] Output: 2 Explanation: As shown in the figure above, node 2 is connected to every other node, so 2 is the center. **Example 2:** Input: edges = [[1,2],[5,1],[1,3],[1,4]] Output: 1 **Constraints:** • 3 <= n <= 10^5 • edges.length == n - 1	Easy

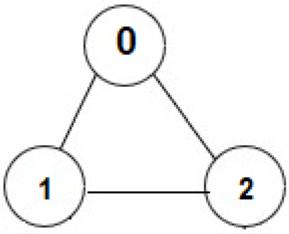
	• edges[i].length == 2 • 1 <= ui, vi <= n • ui != vi The given edges represent a valid star graph.	
4	**Is Graph Tree?** **Problem statement** Gwen gives Ben an undirected graph of 'N' nodes numbered from 0 to 'N - 1' having 'M' edges. The task of Ben is to check whether the graph given by Gwen is a tree or not. Note : There are no parallel edges and self-loops in the graph given by Gwen. A tree is a connected graph having no cycles. **Sample Input 1:** 2 7 6 0 1 0 4 1 2 1 3 4 5 4 6 3 3 0 1 0 2 1 2 **Sample Output 1:** 1 0 **Explanation For Sample Input 1:** Test Case 1 : The given graph is shown below.	Easy



The above graph is a connected graph with no cycles. Hence this graph is a tree. So we need to print 1.

Test Case 2 :

The given graph is shown below.



Above graph is a connected graph but it contains a cycle : 0 -> 1 -> 2 .Hence this graph is not a tree. So we need to print 0.

Sample Input 2:

2
4 3
0 1
0 2
3 2
5 5
0 1
0 3
2 1
2 3
1 4

Sample Output 2:

1
0

	Constraints : • 1 <= T <= 5 • 1 <= N <= 5000 • 1 <= M <= min(5000, N*(N-1)/2) • 1 <= U, V<= N – 1	
5	**Find if Path Exists in Graph** **Problem Statement:** There is a bi-directional graph with n vertices, where each vertex is labeled from 0 to n - 1 (inclusive). The edges in the graph are represented as a 2D integer array edges, where each edges[i] = [ui, vi] denotes a bi-directional edge between vertex ui and vertex vi. Every vertex pair is connected by at most one edge, and no vertex has an edge to itself. You want to determine if there is a valid path that exists from vertex source to vertex destination. Given edges and the integers n, source, and destination, return true if there is a valid path from source to destination, or false otherwise. **Example 1:** Input: n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2 Output: true Explanation: There are two paths from vertex 0 to vertex 2: - 0 → 1 → 2 - 0 → 2 **Example 2:** Input: n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5 Output: false Explanation: There is no path from vertex 0 to vertex 5. **Constraints:** • 1 <= n <= 2 * 10^5 • 0 <= edges.length <= 2 * 10^5 • edges[i].length == 2 • 0 <= ui, vi <= n - 1 • ui != vi • 0 <= source, destination <= n - 1	Easy

	• There are no duplicate edges. • There are no self edges.	